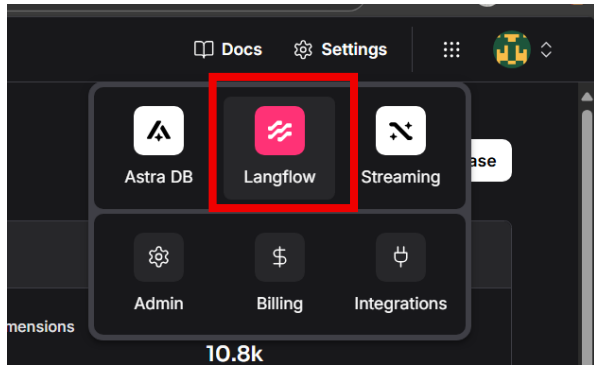
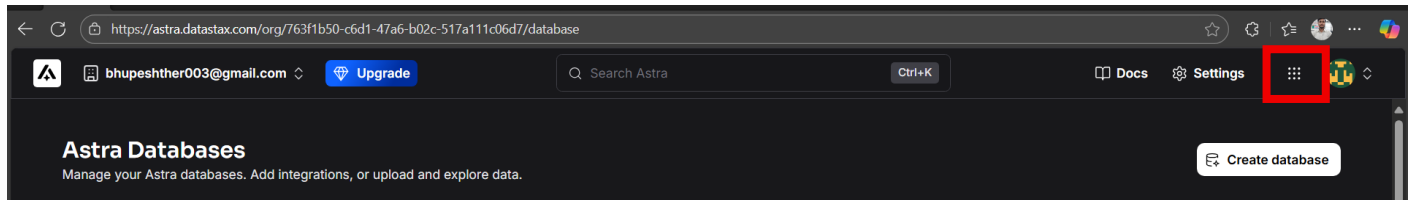
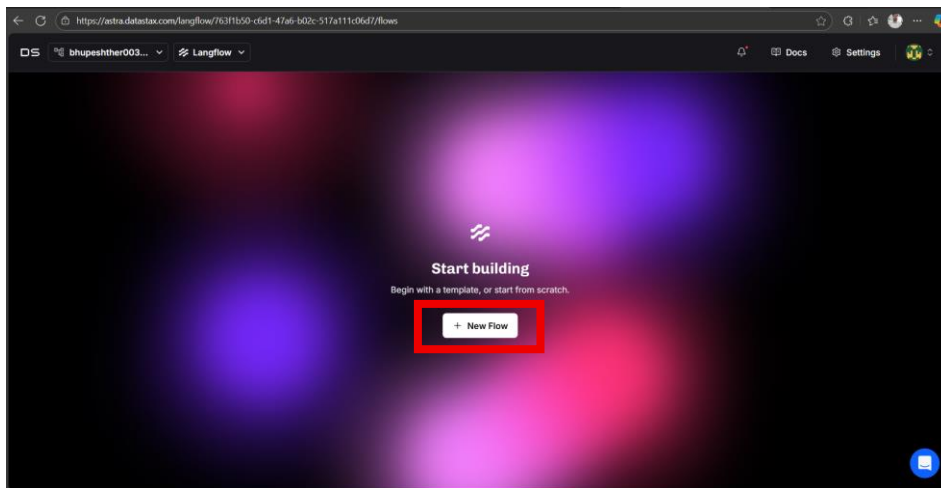


RAG agent with Astra DB using watsonx.ai

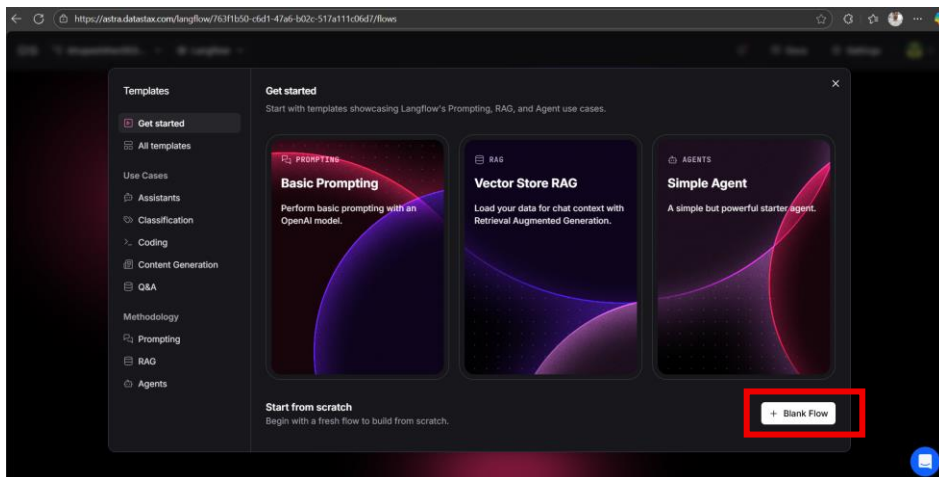
Create the astra DB account and click on right side square box and select langflow



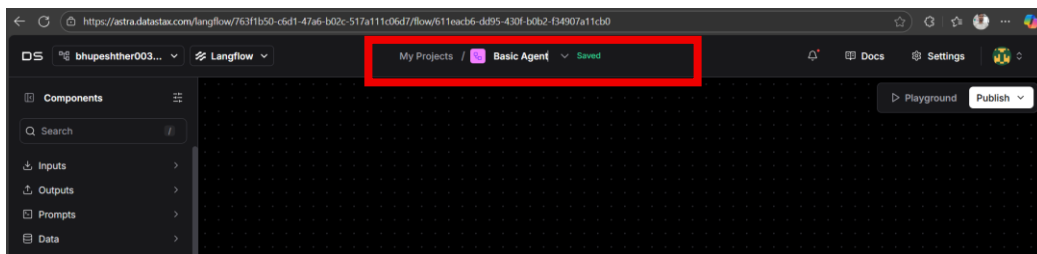
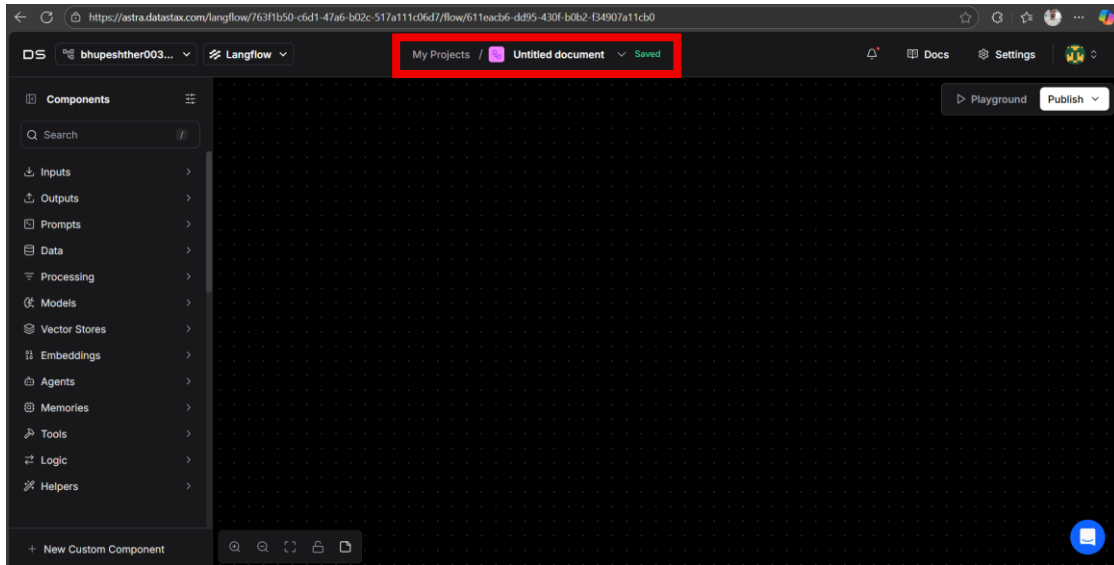
1. Open the langflow online.



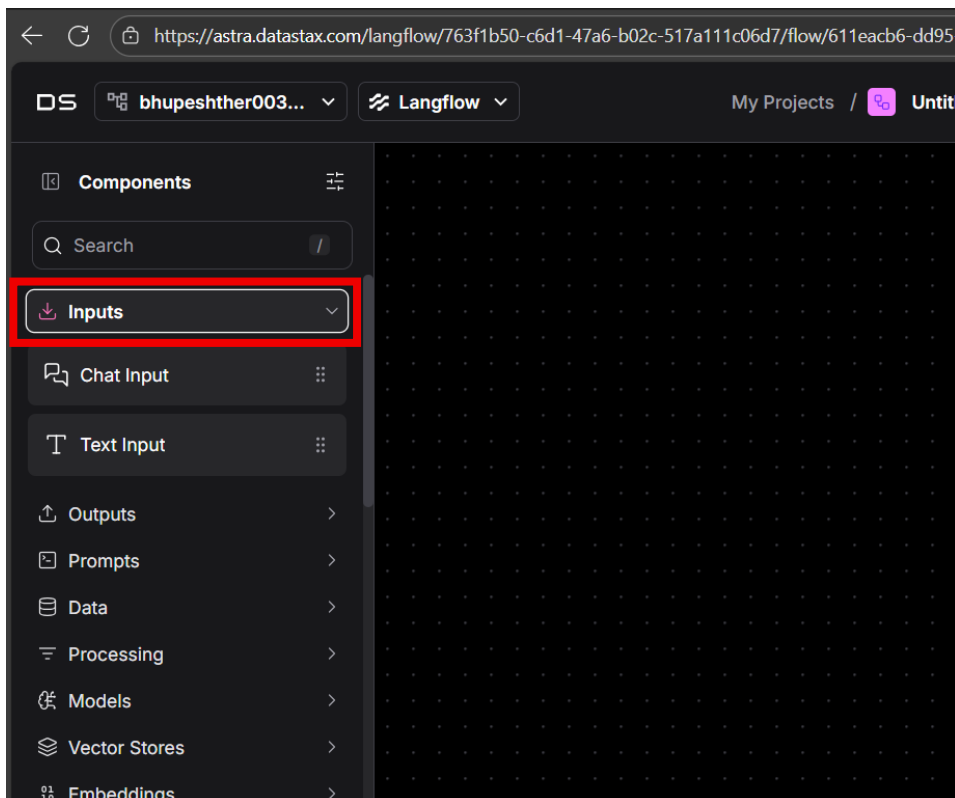
2. We need to create one project for this click on the **blank flow**



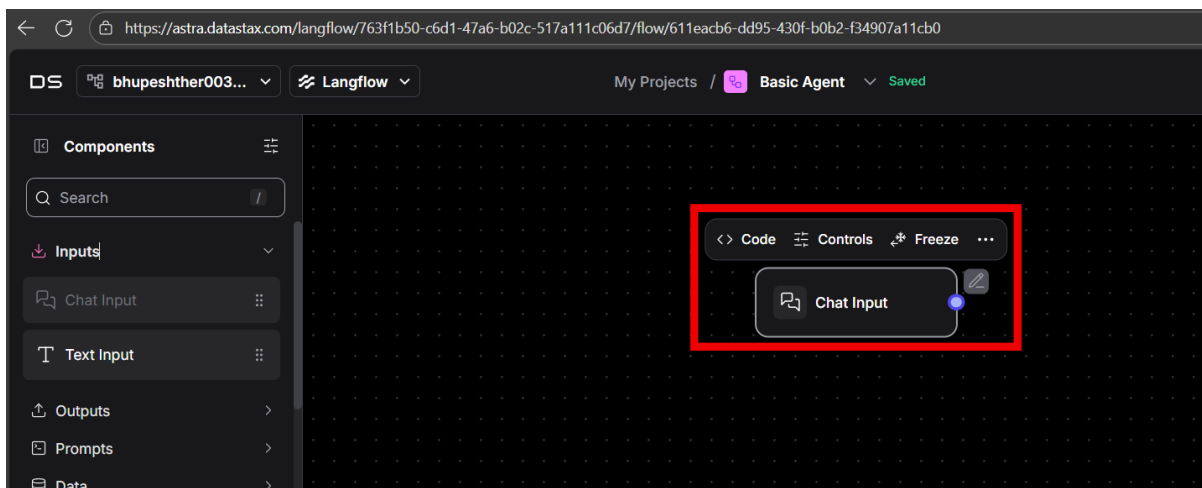
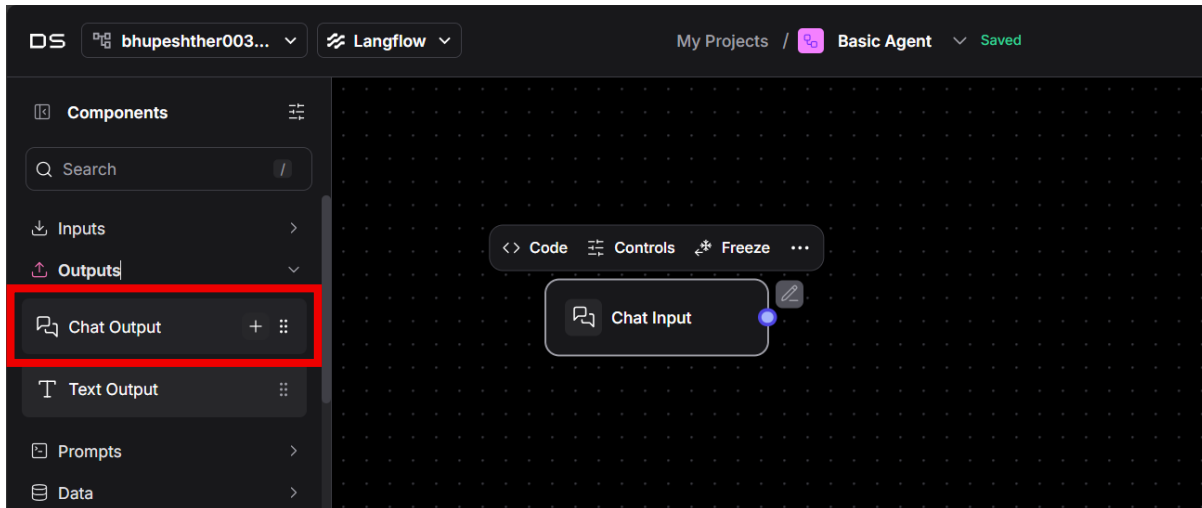
3. Now you are in the workspace where we are creating the our Agent. Change the agent name for this click one untitled document and change the name of your project



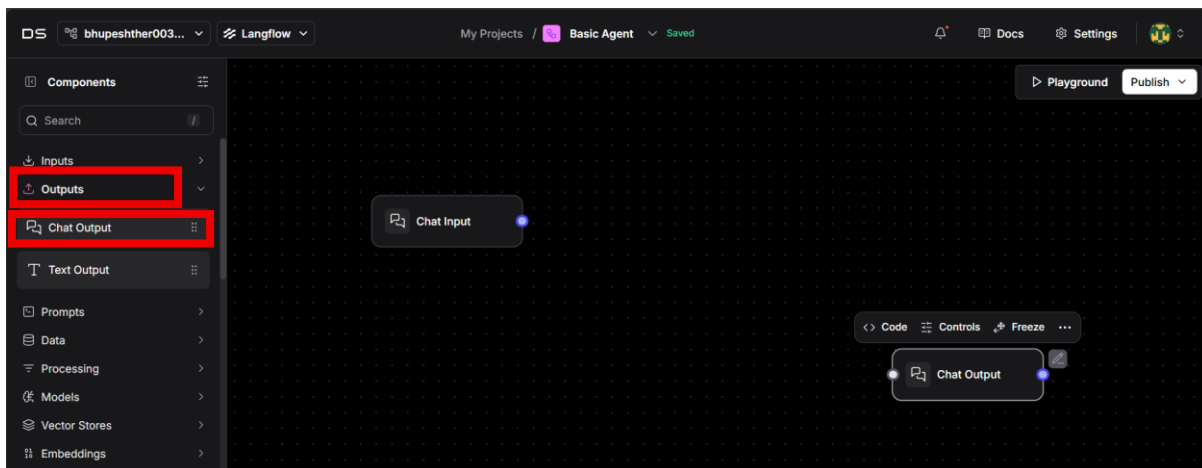
4. Before the starting we need to add input chat so click on the inputs and form that select the chat input.



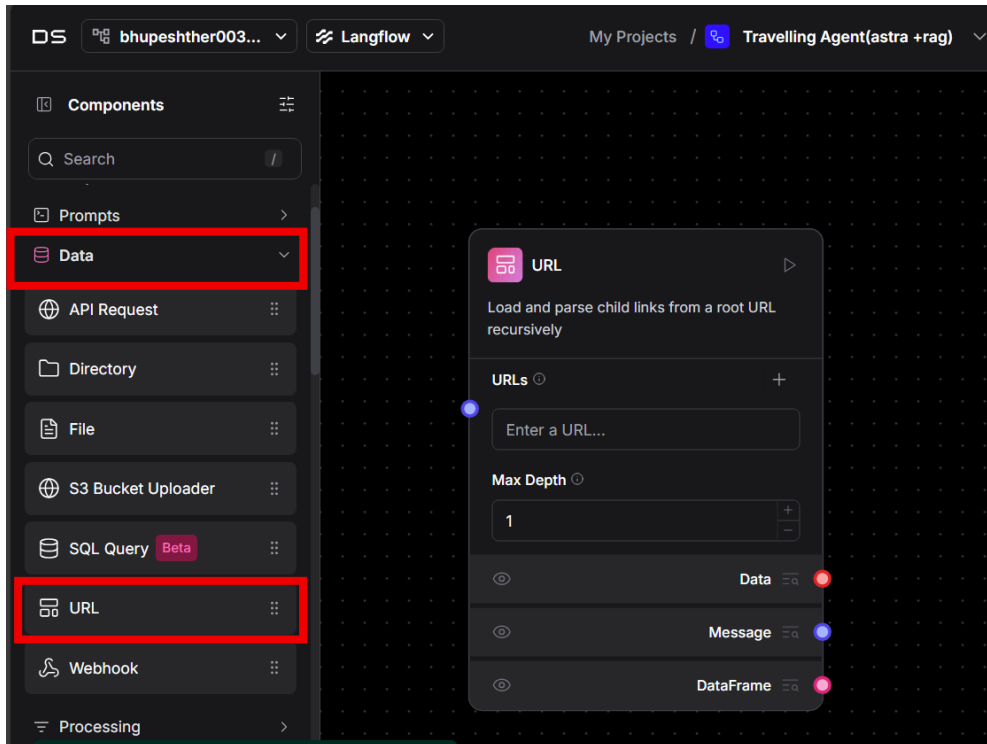
5. Click in + icon to add in the workspace (also you can do drag and drop)



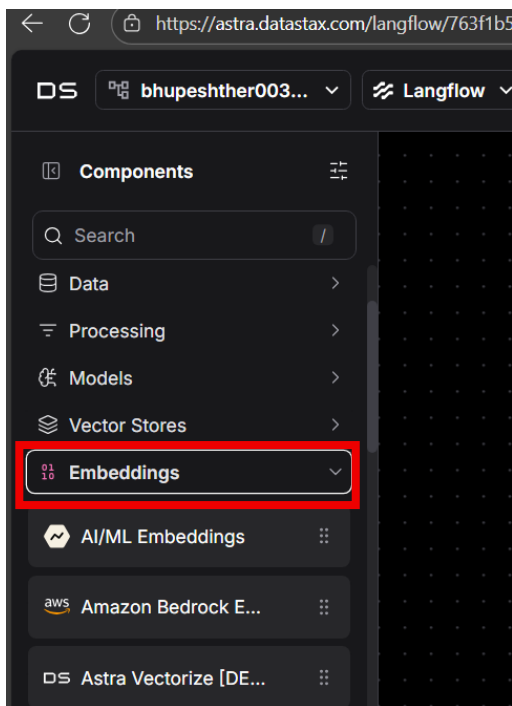
6. Same we need to add output as well so click on the output and select the chat output .



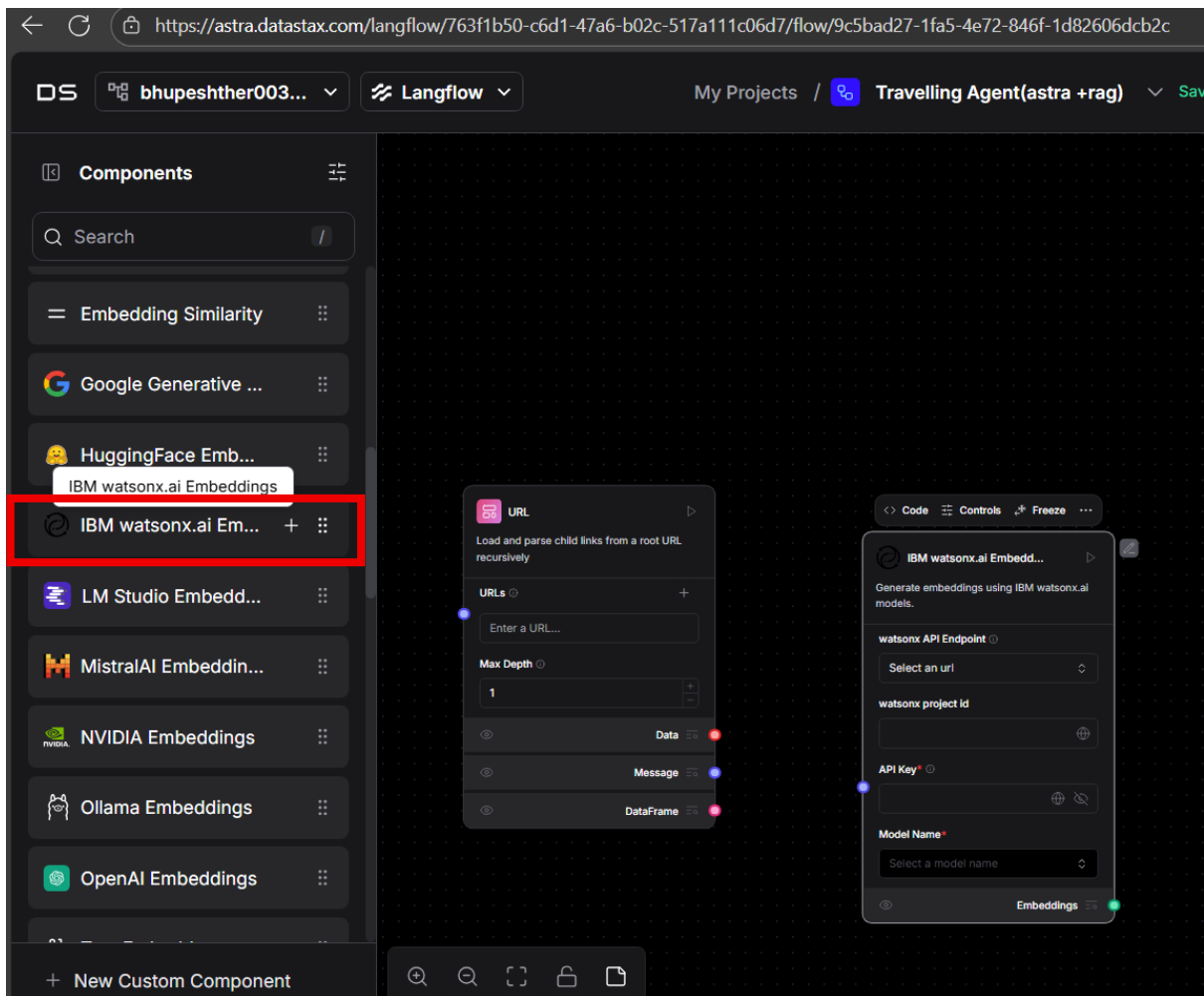
7. So u need to add url for adding the website to get up-to date information.



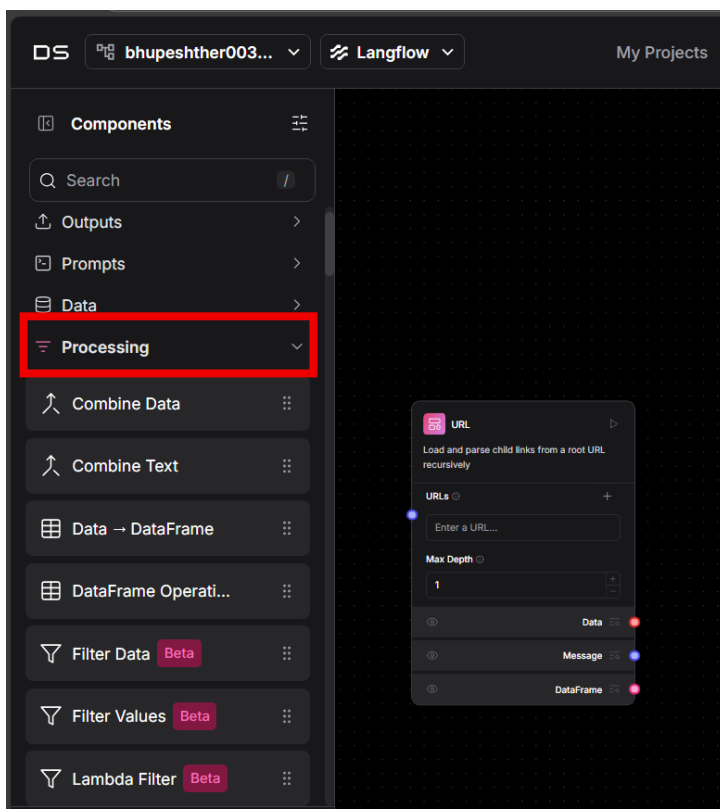
8. So we need to add embedding For that need to add embedding model.



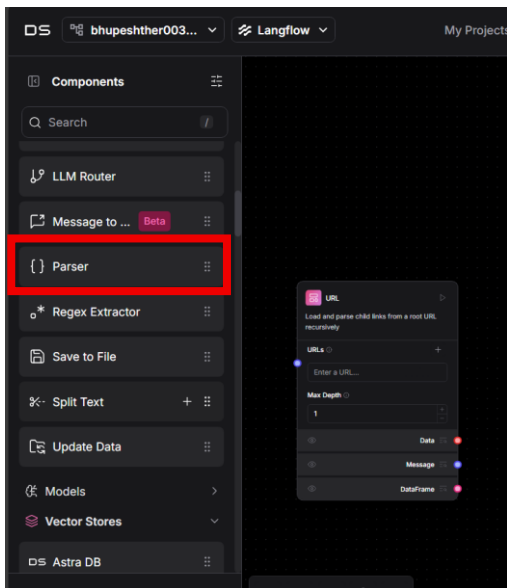
9. Form the list From the list we need to select IBM Watson.ai Embeddings



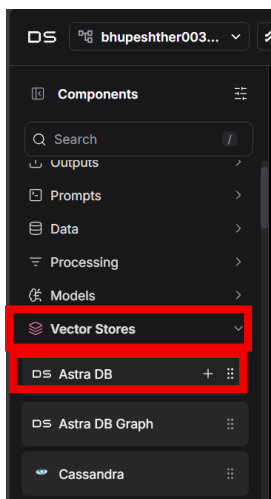
10. Now next for the parse go to the Preprocessing



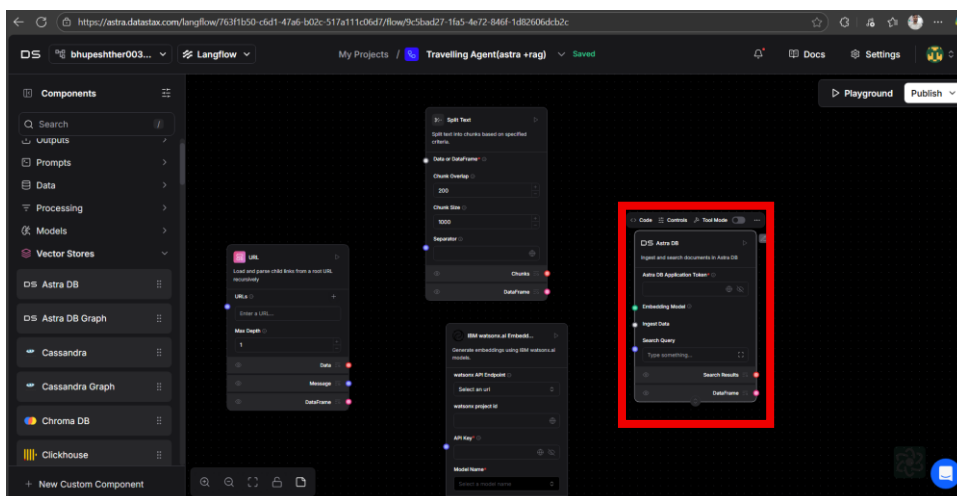
11. And from this select the parse



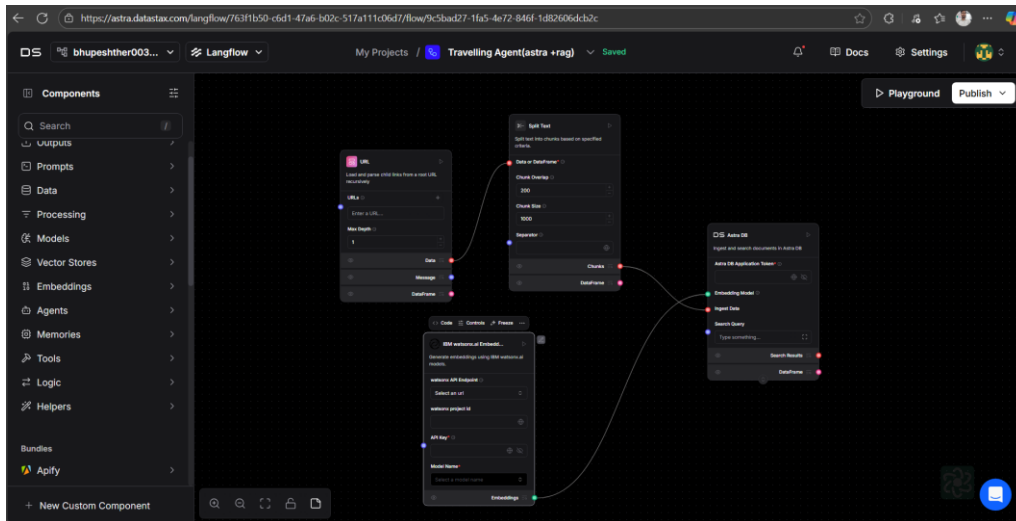
12. Now we need to select the Astra DB to store our information for this go to the vector stores and select Astra DB



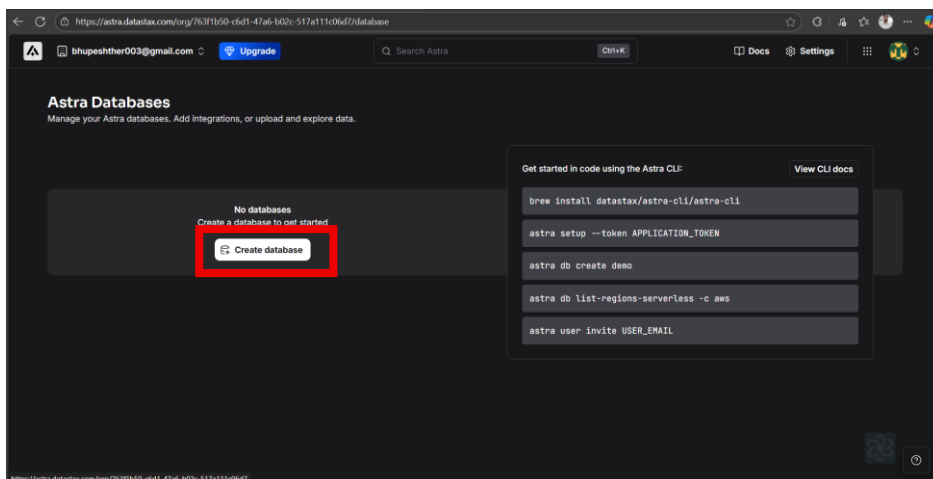
13. Now using drag and drop add in the workspace



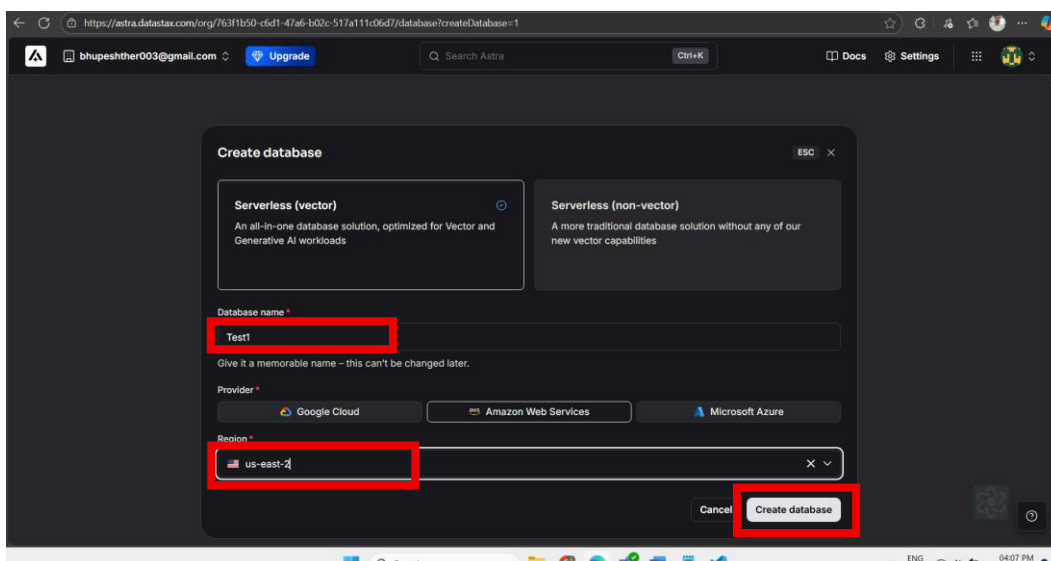
14. Now make connection for this follow below diagram



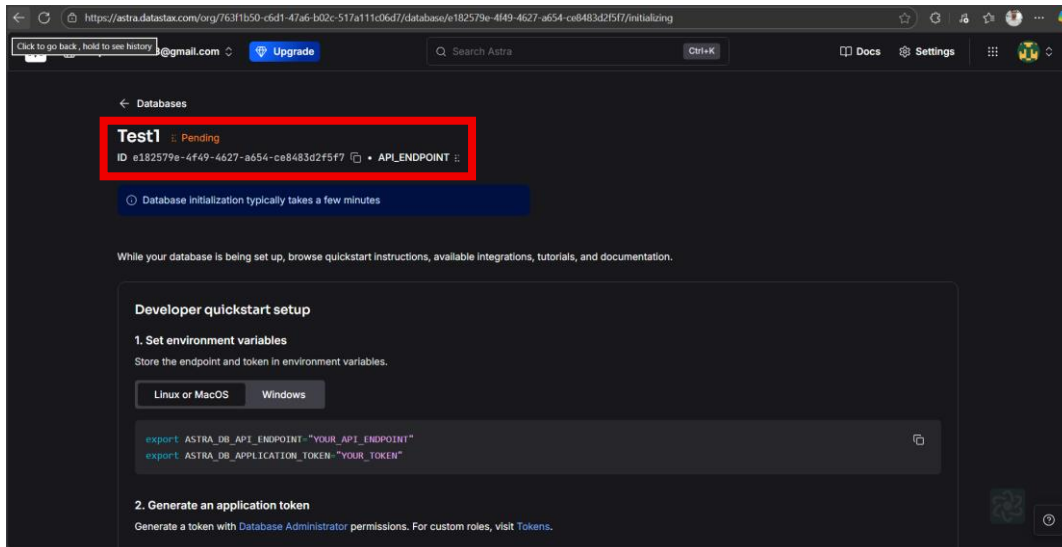
15. Once its done now go to the astra db to create Vector database and click on the create database option



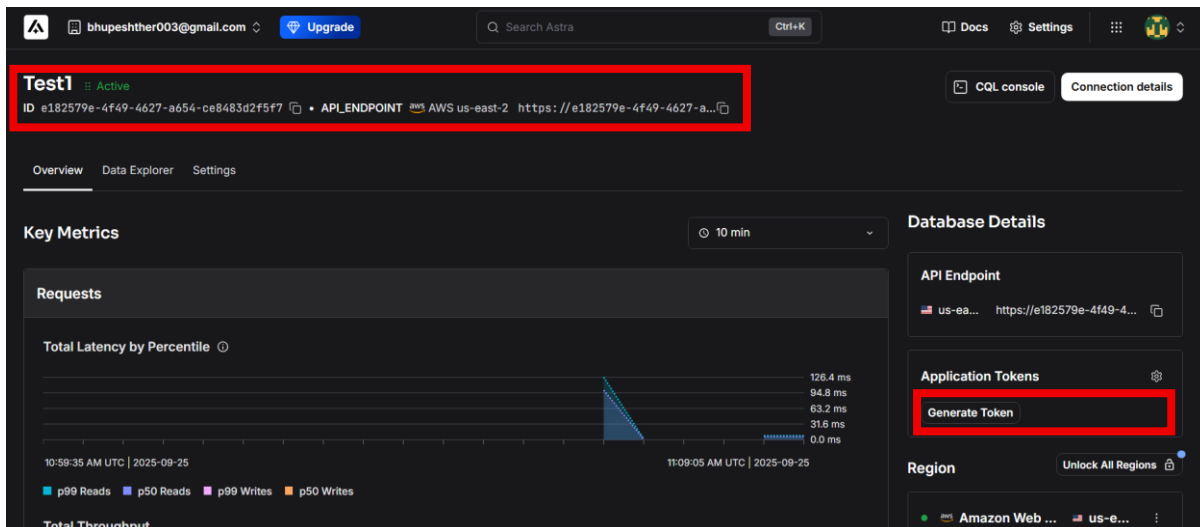
16. Give the name of the database and select region as **us-east-2** and click on the create



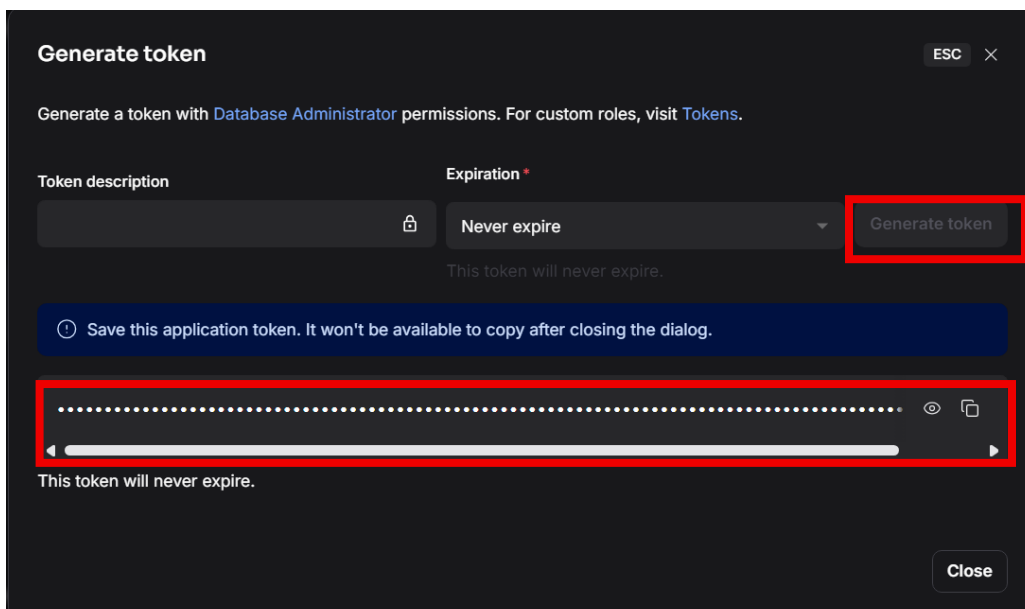
17. Now its will take couple to time to create the database wait for while



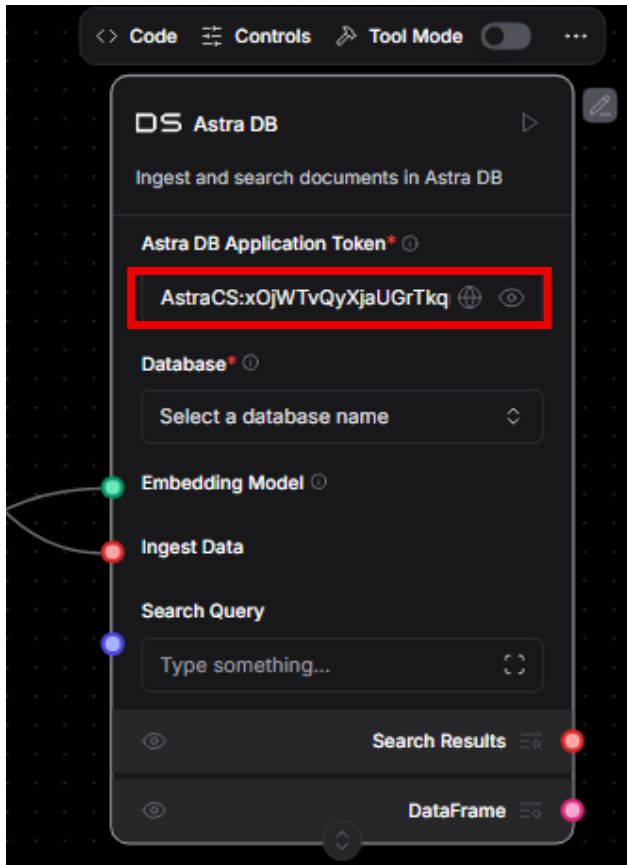
18. Once its done in your right side you getting the option for **Generate Token** click on that



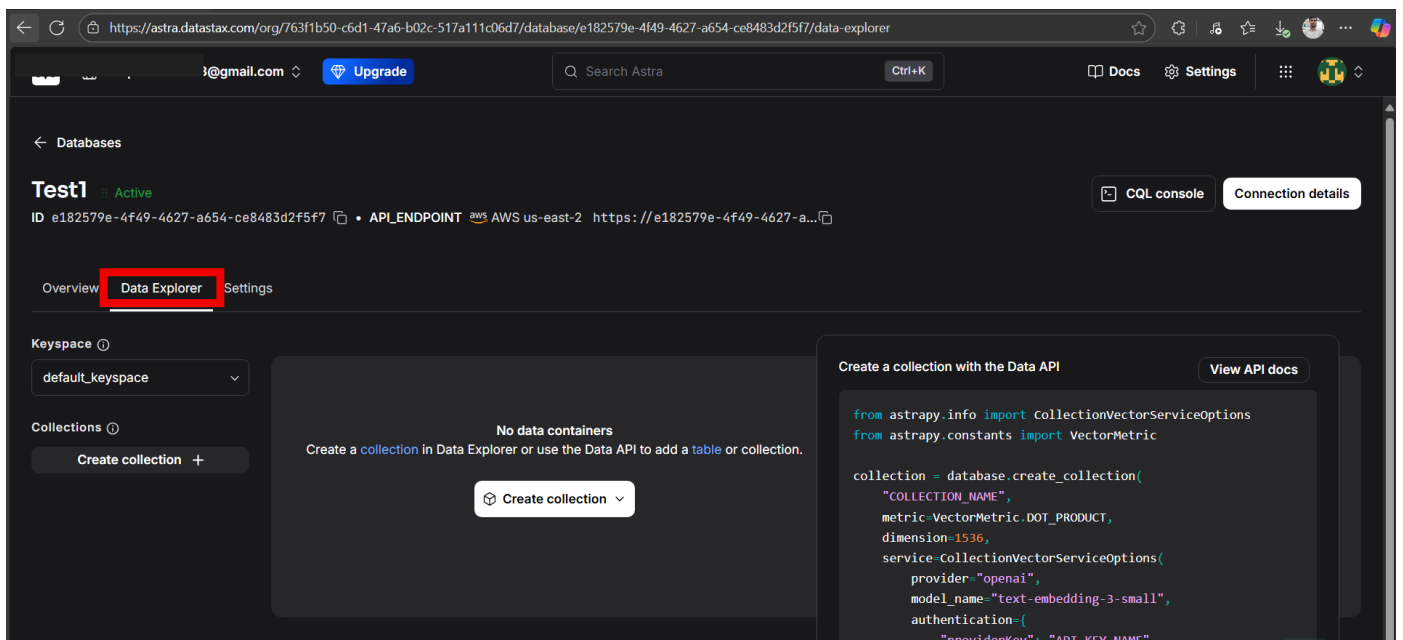
19. Click on generate token and your token will be generated of that one and paste somewhere because in hole project we need that one



20. Now come to langflow and and that token in the Astra DB Application Token



21. Once its done come back to Astra DB and now create one collection over here for this click on create collection button



22. Give the name of **collection** and embedding generation method as **bring my own** and dimensions in **768** and click on create collection

Create collection [ESC] [X]

Collection name *
test_tatble

☒ Vector-enabled collection ⓘ

Embedding generation method *
Bring my own

Select a configured embedding provider integration or generate your own embeddings

Dimensions * ⓘ
768
The number of components in a single vector

Similarity metric * ⓘ
Cosine
Method used to calculate vector similarities

By creating a collection, you agree to the [DataStax Preview Terms](#).

Cancel Create collection

23. Once its done Now go back on langflow and select the database and after that select the Collection and you created

Astra DB

Ingest and search documents in Astra DB

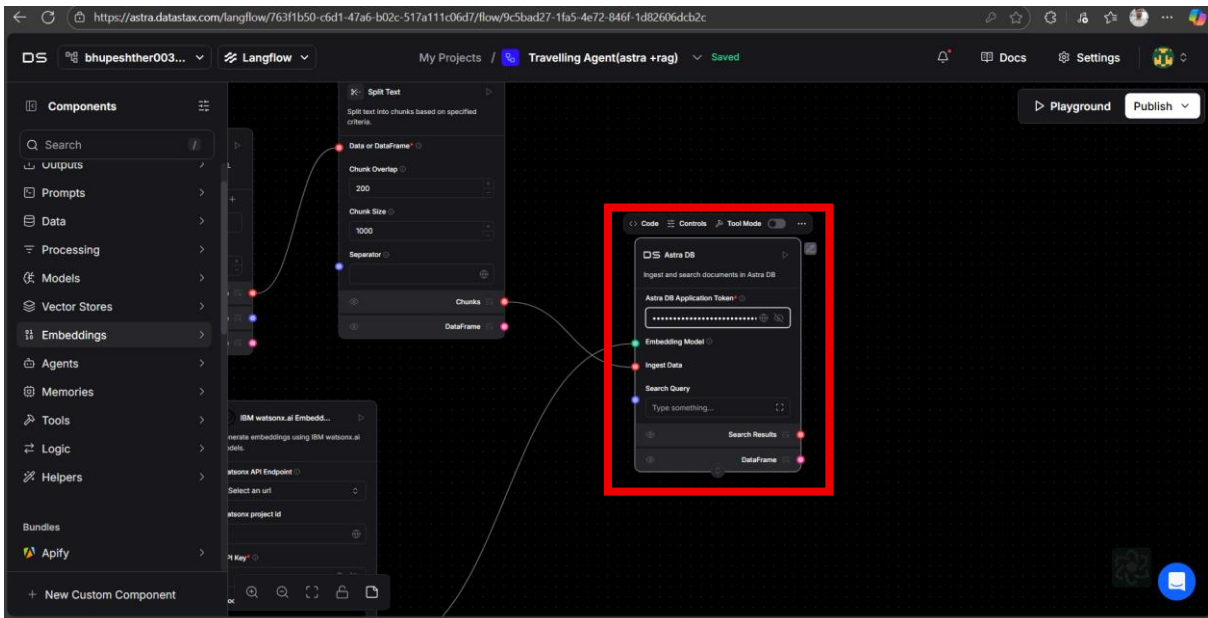
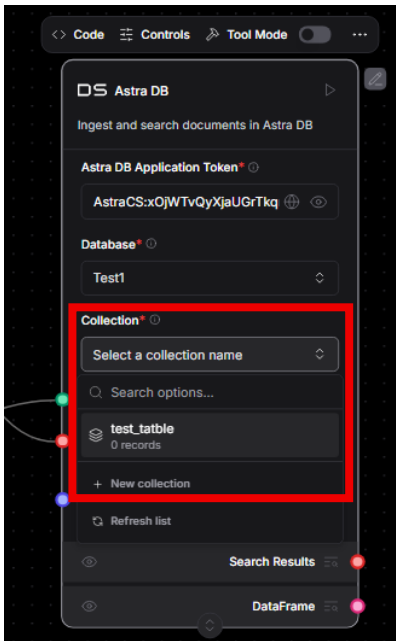
Astra DB Application Token * ⓘ
AstraCS:xOjWtVqyXjaUGrTkq

Database * ⓘ
Select a database name
Search options...
Test1
+ New database
Refresh list

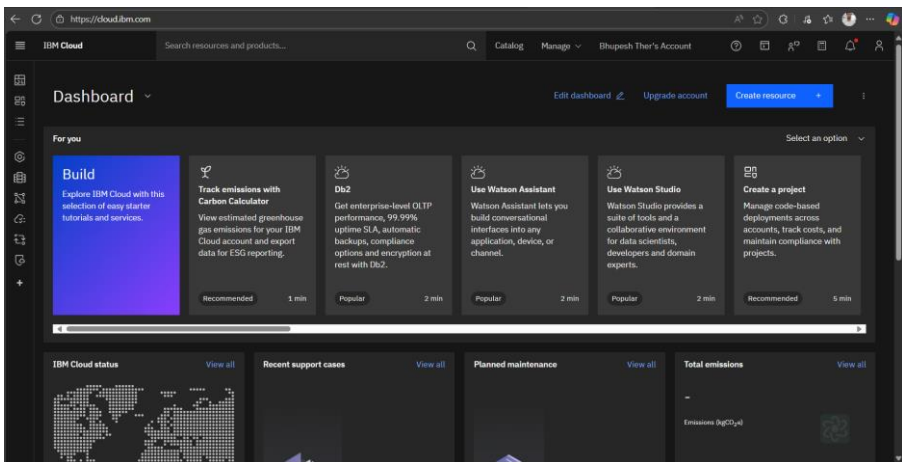
Search Results

DataFrame

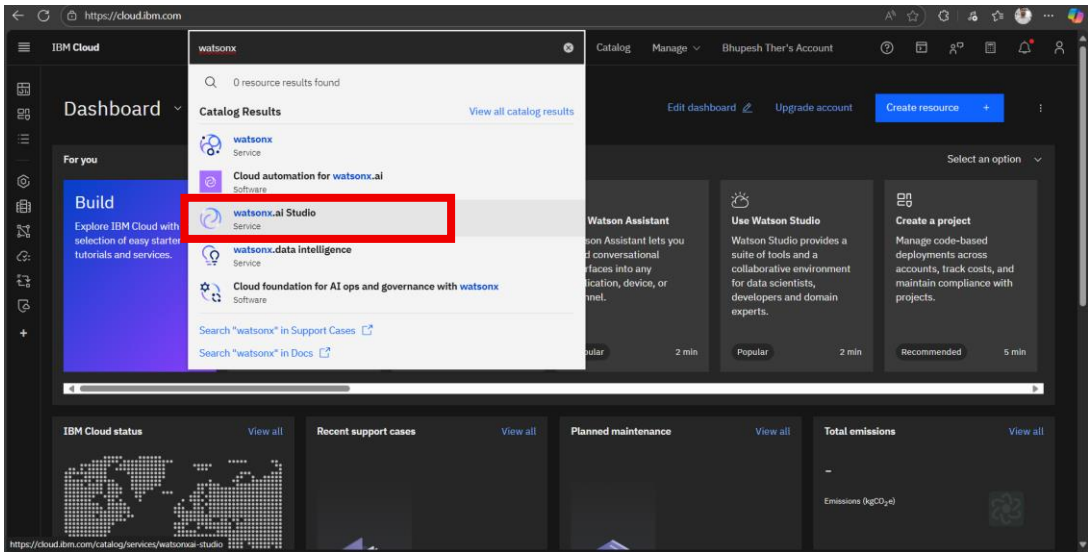
Final setup looks like below image



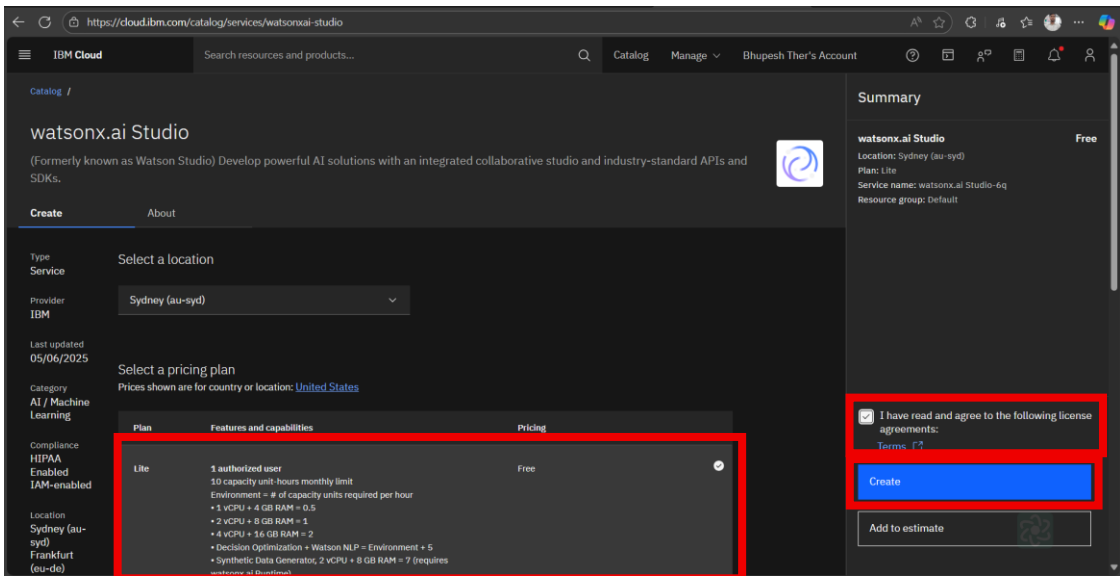
24. Now next for the embedding model we need project id and api for this go to ibm cloud



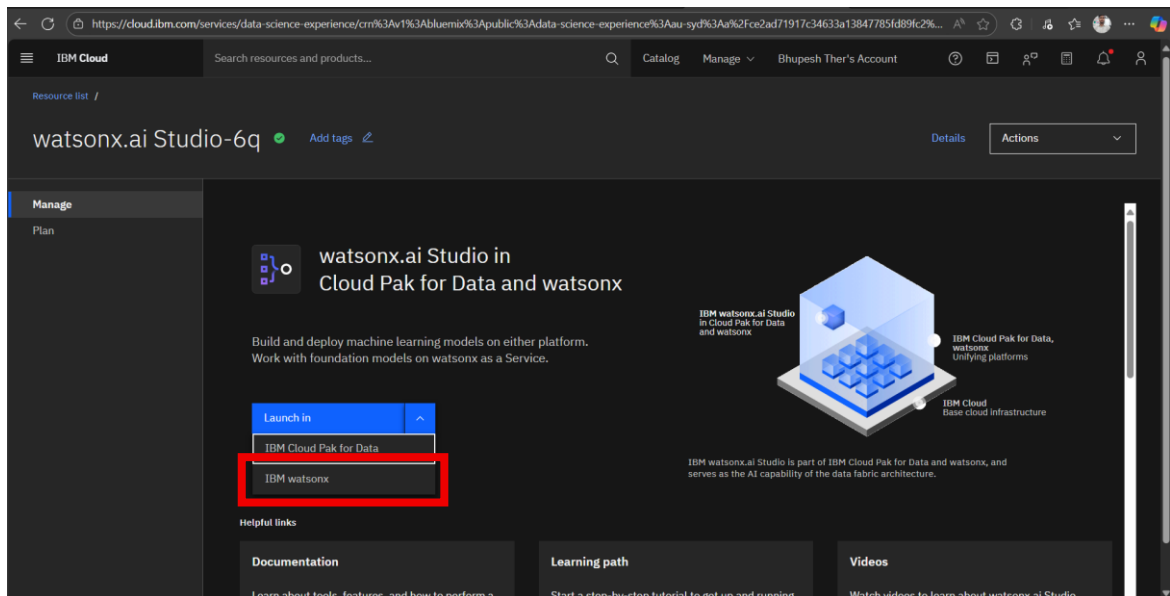
25. Go to search bar and search for the IBM watsonx.ai and click on that



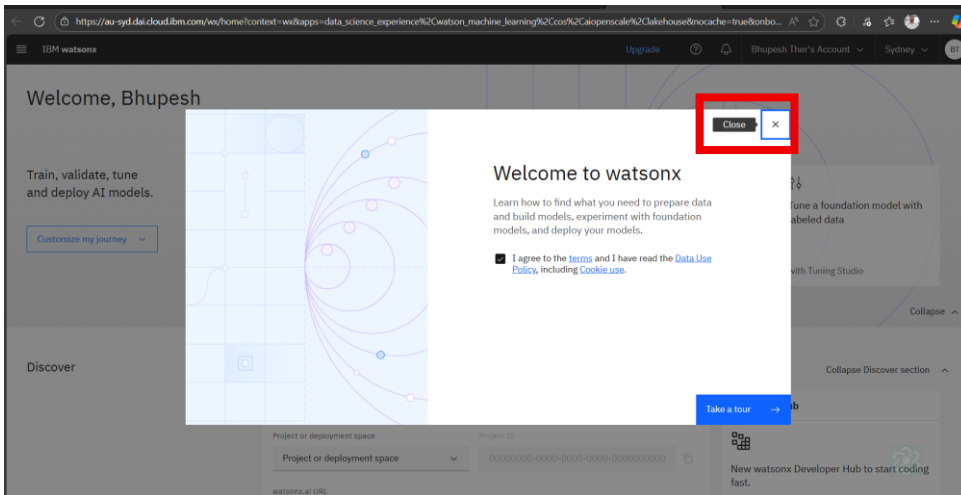
26. Now you need to create IBM watsonx service so make right side [] square box and click in the create



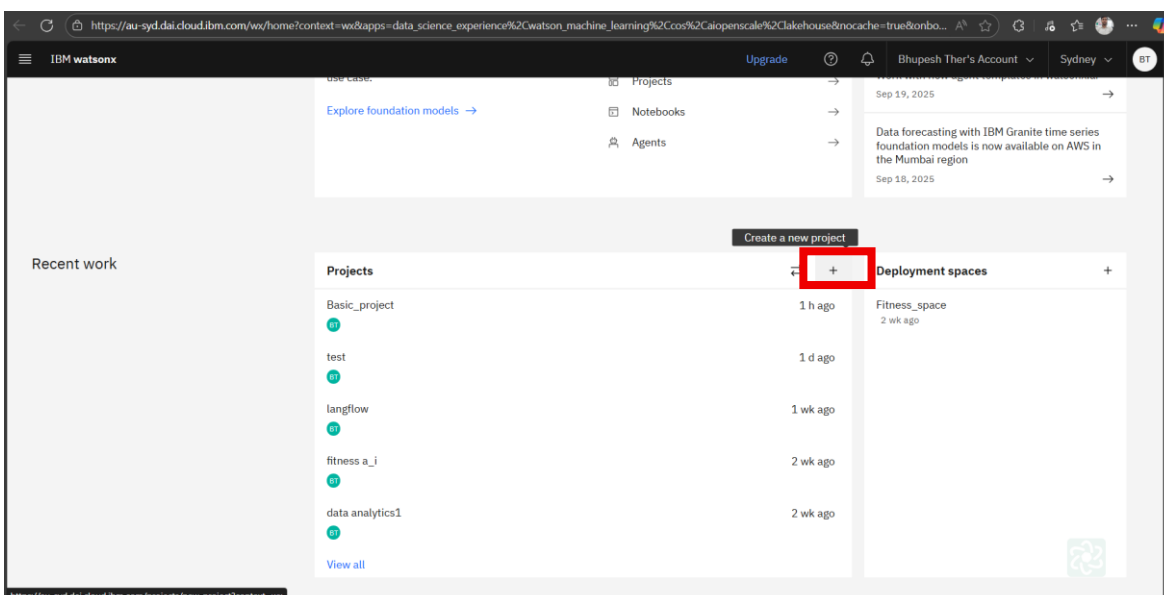
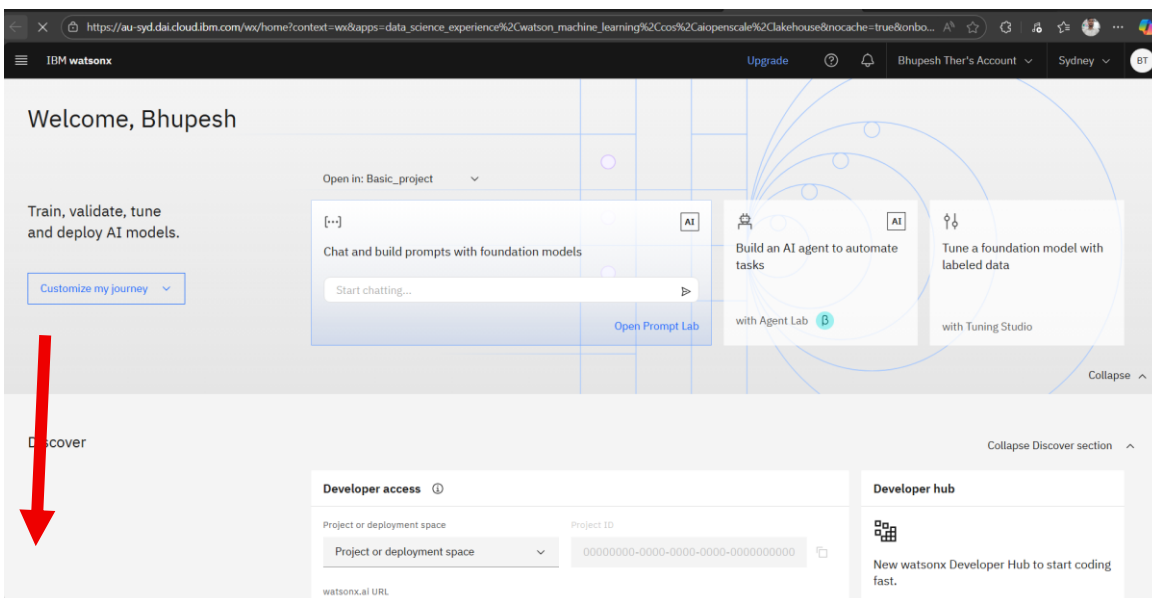
27. Once its done they will redirect you on the lunning page and click on the dropdown and select IBM watsonx



28. Now you will this on this page click on the X cross arrow and close this tour



29. Scroll up this page little bit more and come to projection section and click on the + icon and create new project



30. Give the project name and click on the create

IBM watsonx

Create a project

Start with a new, blank project or select from where to import an existing project.

+ New

Local file

Sample

Name

basic_project_1

Description (optional)

What's the purpose of this project?

Tags (optional)

Add tags

Add tags to make projects easier to find. To add tags, separate them with commas and press Enter.

Storage

Cloud Object Storage-kk

Project includes integration with Cloud Object Storage for storing project assets.

Cancel

Create

31. Now you need to go manage

IBM watsonx

Projects / basic_project_1

Overview Assets Jobs **Manage**

Start working

Add users as collaborators

Add data to work with

Chat and build prompts with foundation models

Tune a foundation model with labeled data

View all

Jump back in

Assets that you create with tools show here. See all assets, including data assets, on the Assets page.

View all

Resource usage

For this month in this project

0 CUH

0 Tokens

0 Hosting hours

Your documentation

Get started with your documentation

You can create and manage documents about work that you do in this project.

Open Documentation editor

Project history

32. Now click on services & integrations

IBM watsonx

Projects / basic_project_1

Overview Assets Jobs **Manage**

Project

General

Access control

Environments

Resource usage

Services & integrations

Tools

Pipeline

General

Details

Name

basic_project_1

Description

What's the purpose of this project?

Tags

Add tags to make projects easier to find.

Project ID

a5d253f8-38e3-49ef-9220-f9754c7bfe81

Storage

Storage used

0 Bytes

Bucket

basicproject1-donotdelete-pr-rir5ukl9ryquoo

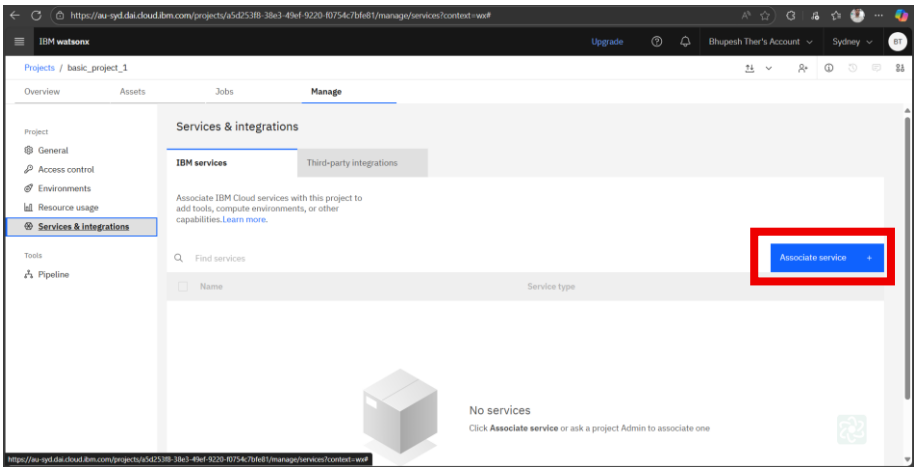
Manage in IBM Cloud

Controls

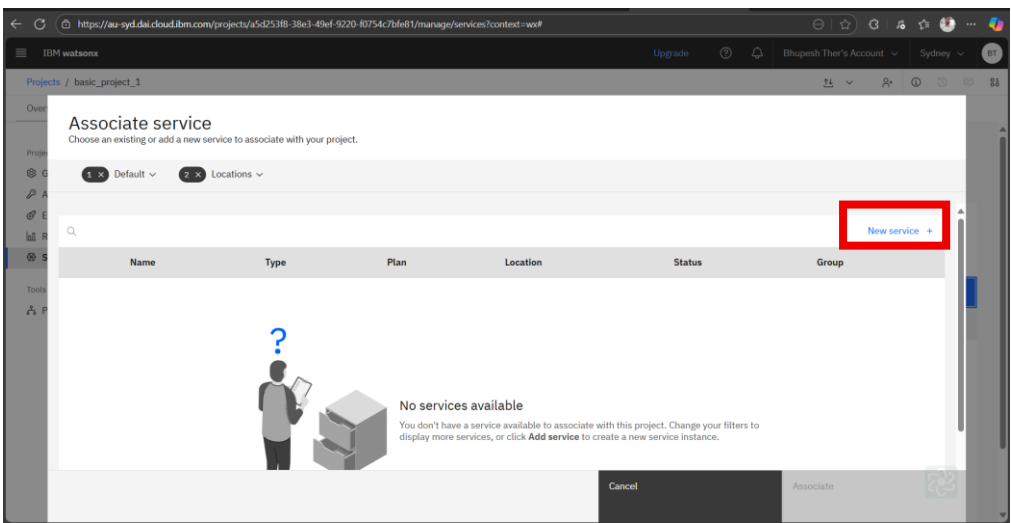
Opt-in to folders

Enable folders

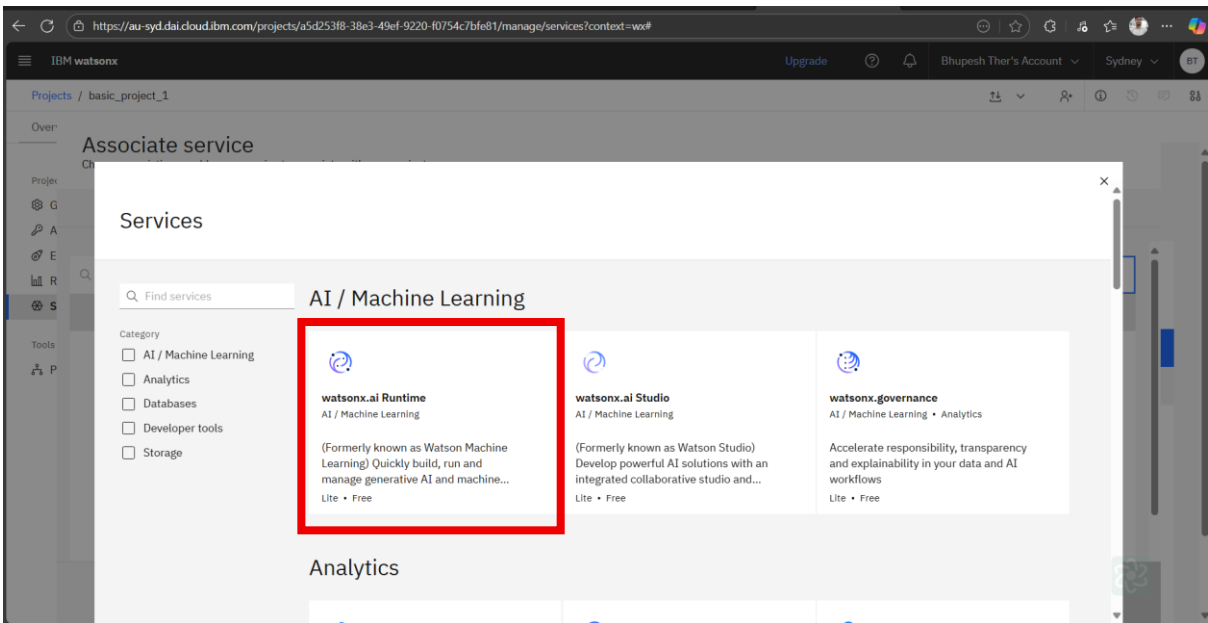
33. Click on the associate service



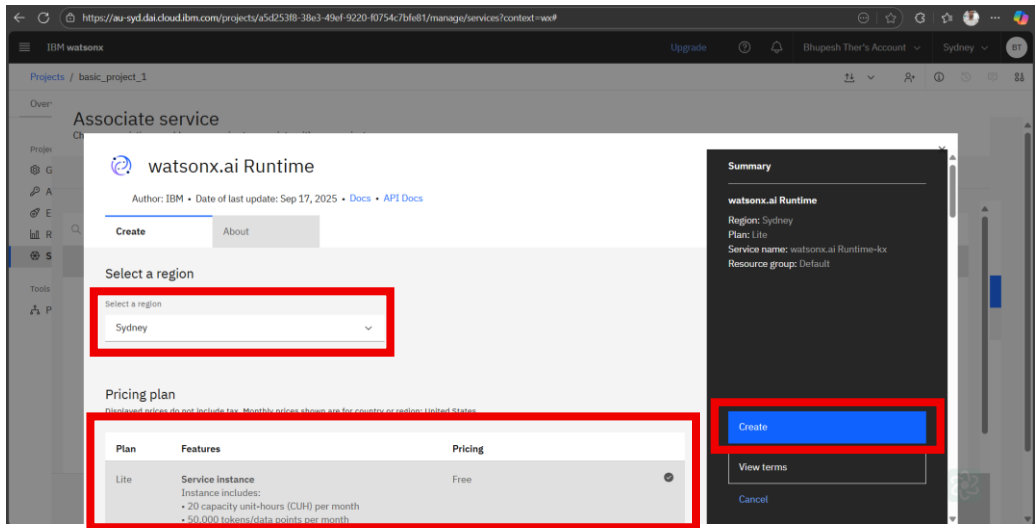
34. Click on new Service



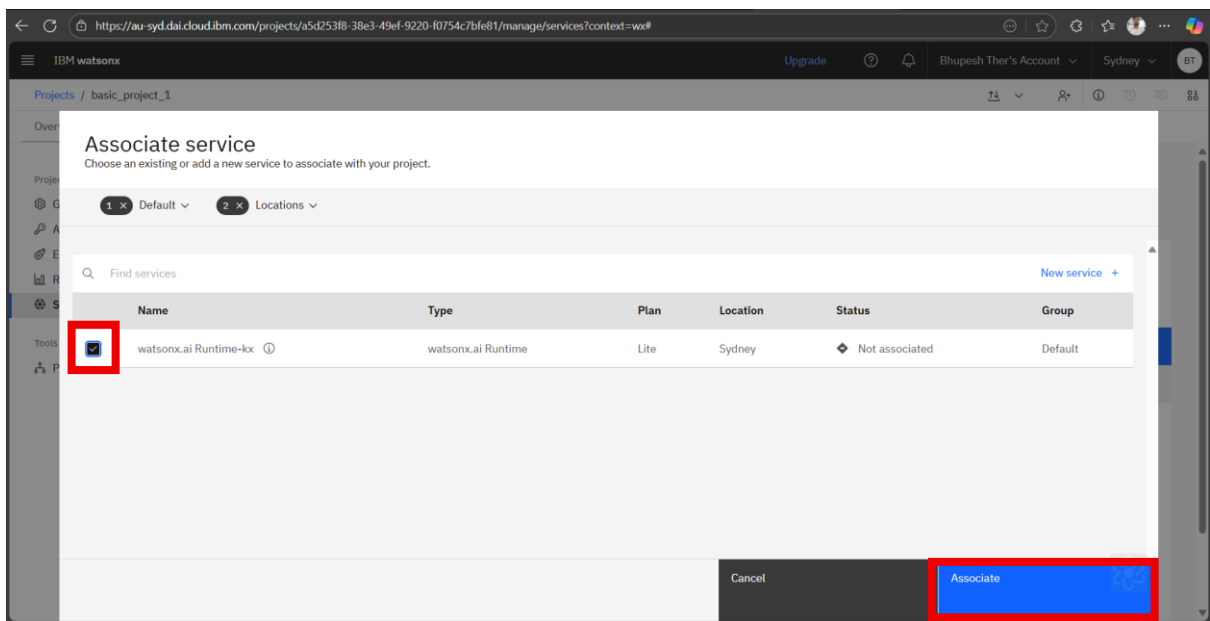
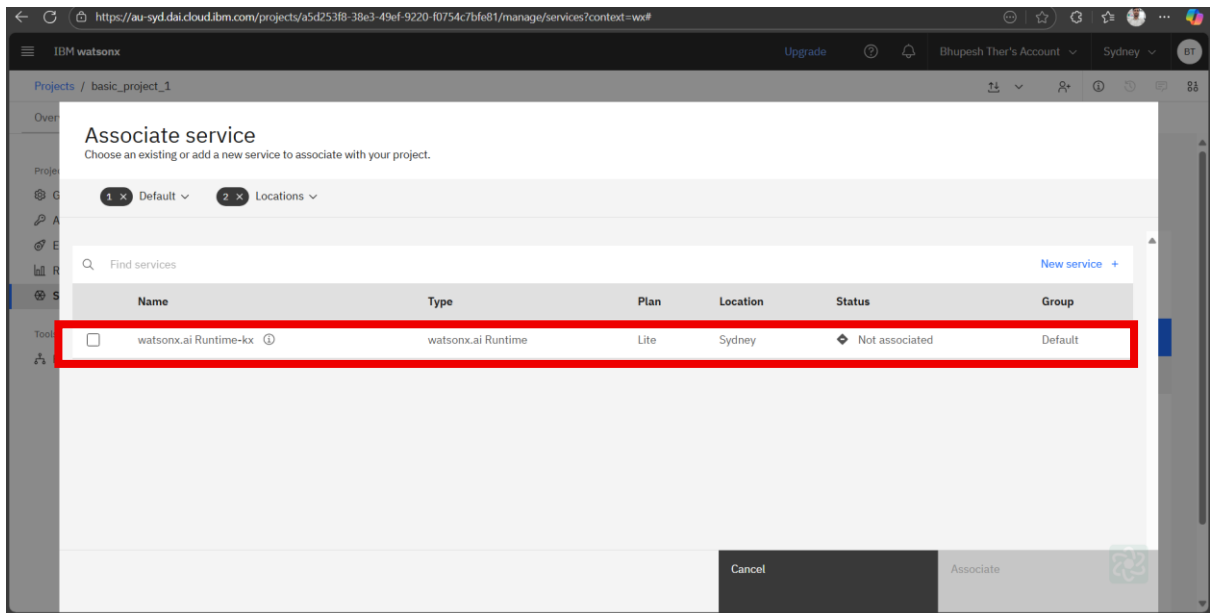
35. Now click on watsonx.ai runtime



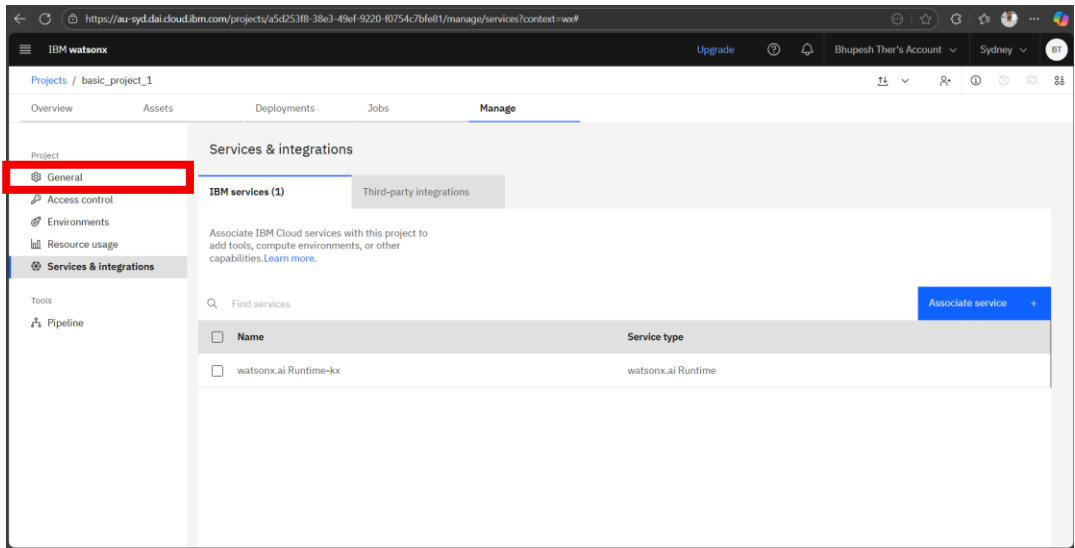
36. Assing the location and select lite plan and click on create



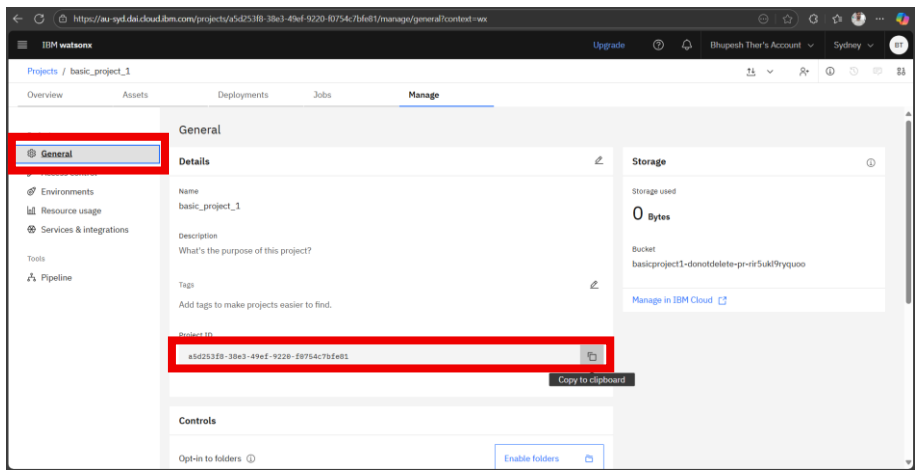
37. Once service created mark check box and click on associate



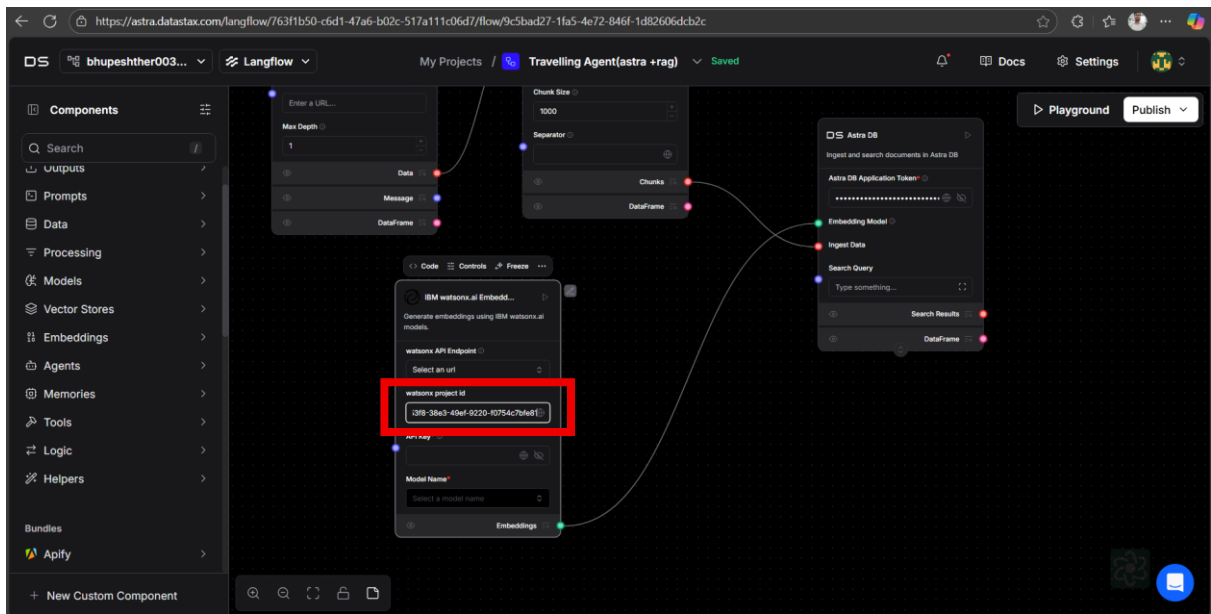
38. Once its done u will be able to see in services & integrations section



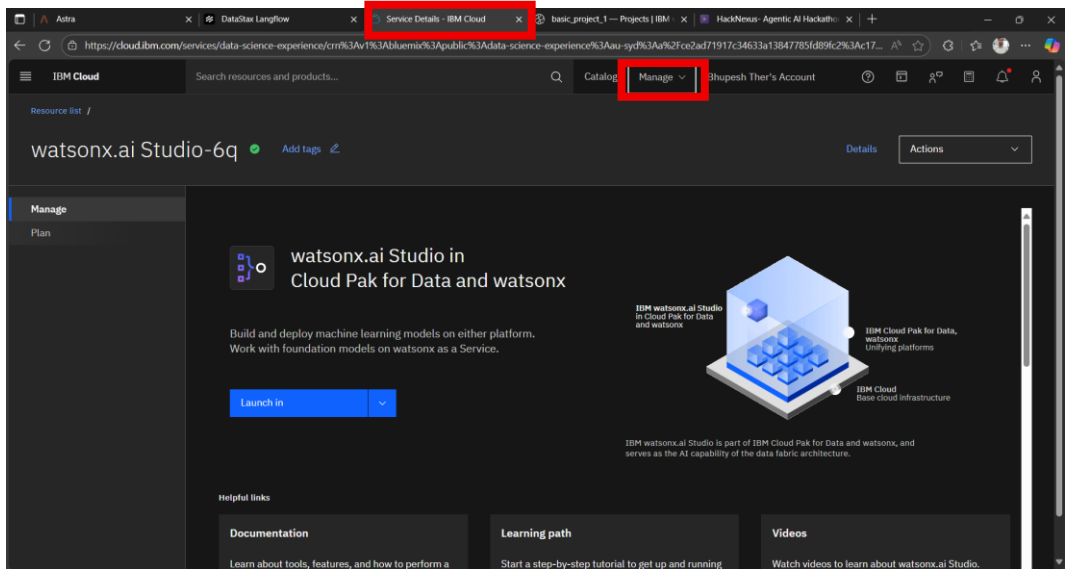
39. Now click on the general here you getting your project id copy your project ID its import so please paste somewhere



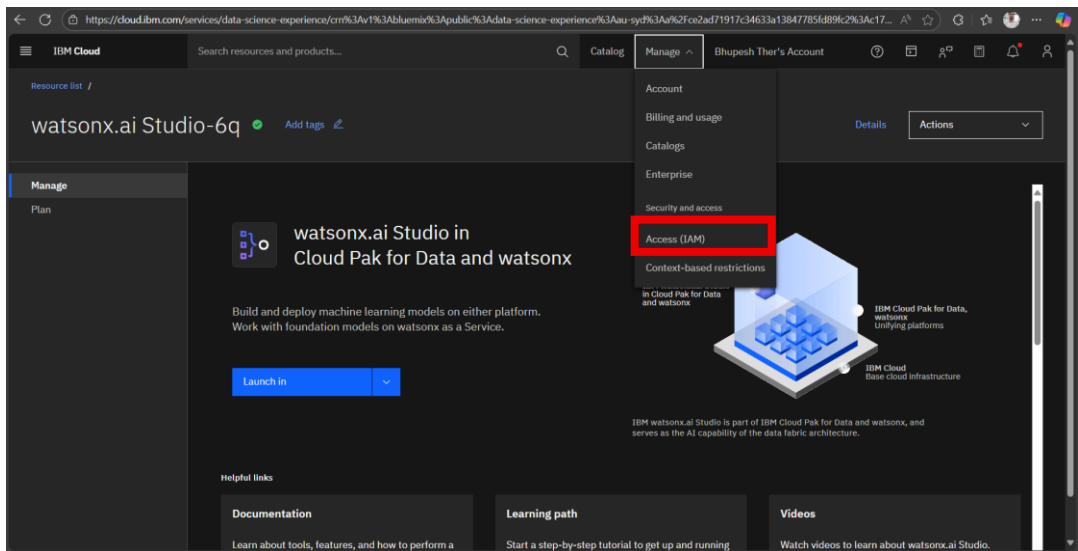
40. Now come back to langflow and add your project ID in Watsonx project ID



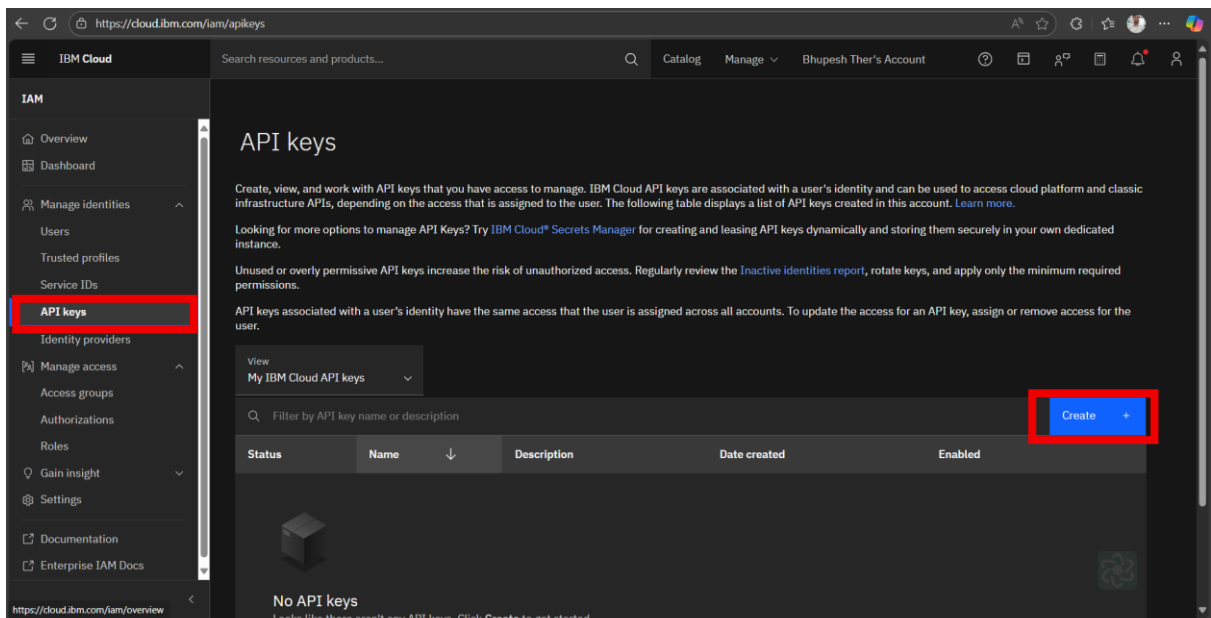
41. Once its done go back to your previous tab that name service details (launching page)



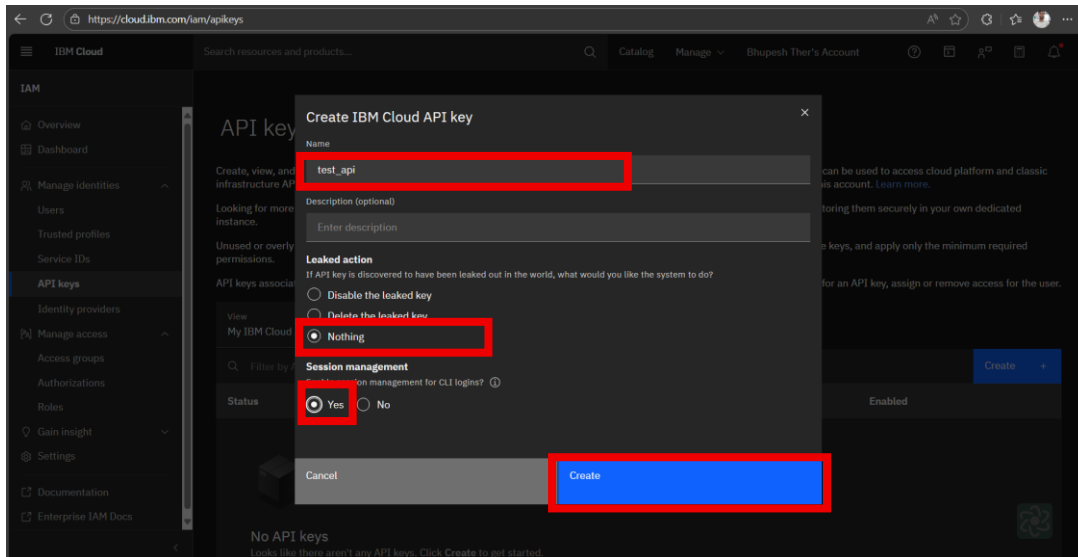
42. Click on manage and next Access(IAM)



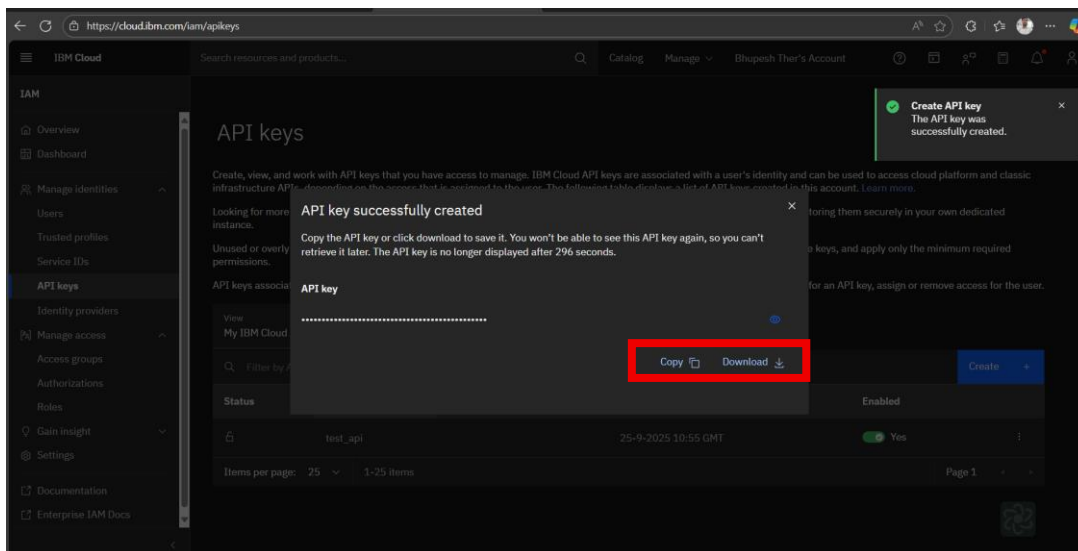
43. Now left side you getting the option of API Key click on that and after that on right side you will be getting option create click in that



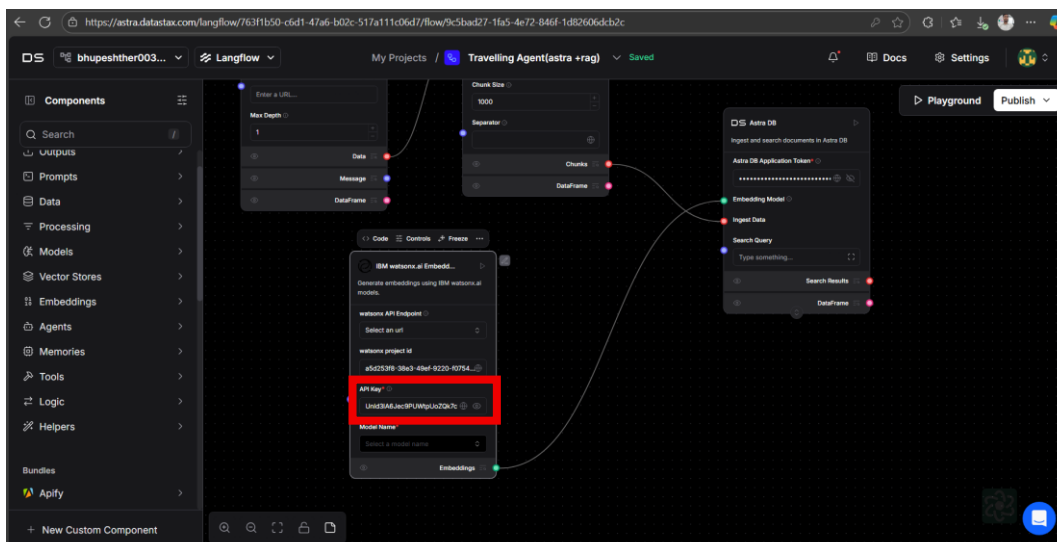
44. Give the name to your API key select leaked action nothing and session management yes and click on create



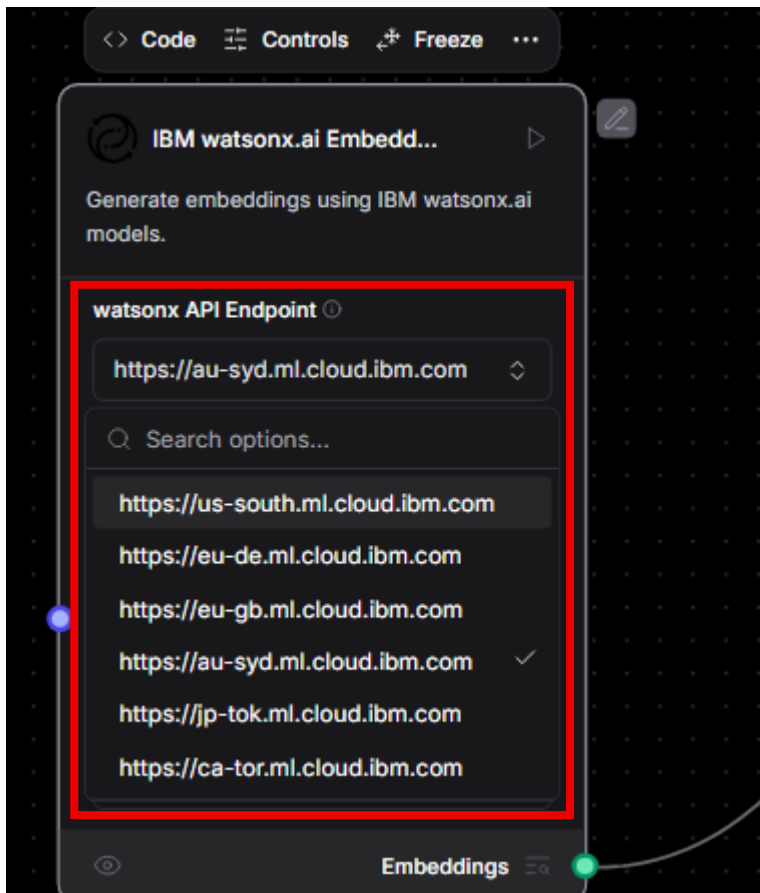
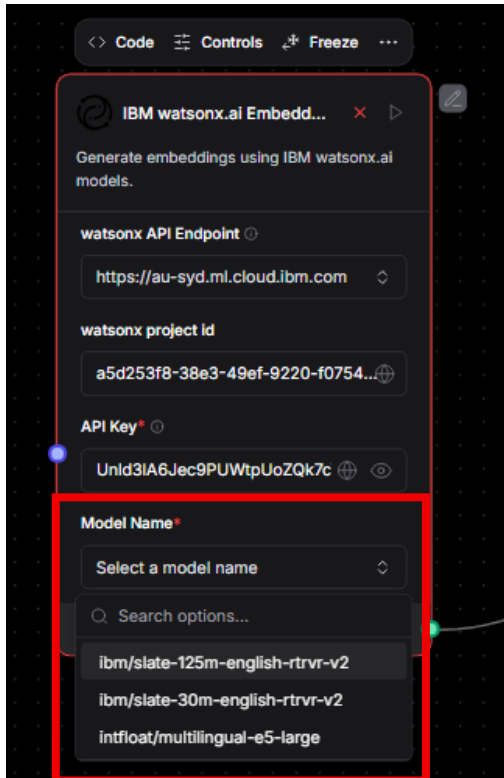
45. Once successfully created now you can copy your api also you can download in your system and its important so please paste somewhere

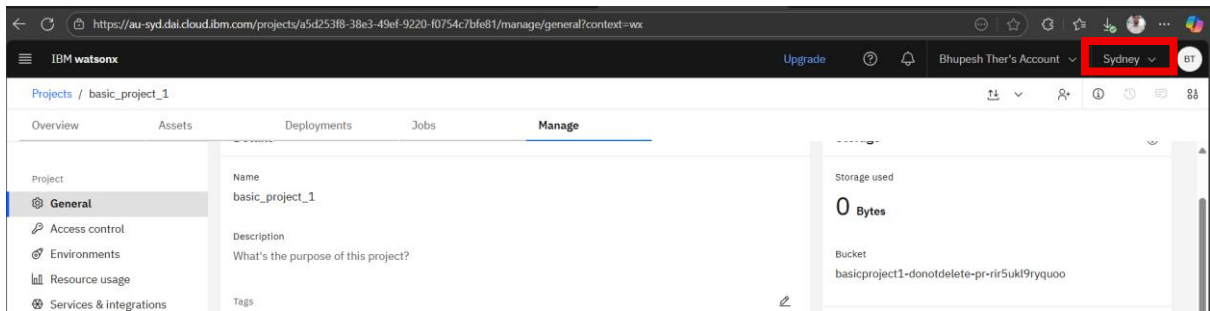


46. Once again back to langflow and now in Watsonx embedding and API key

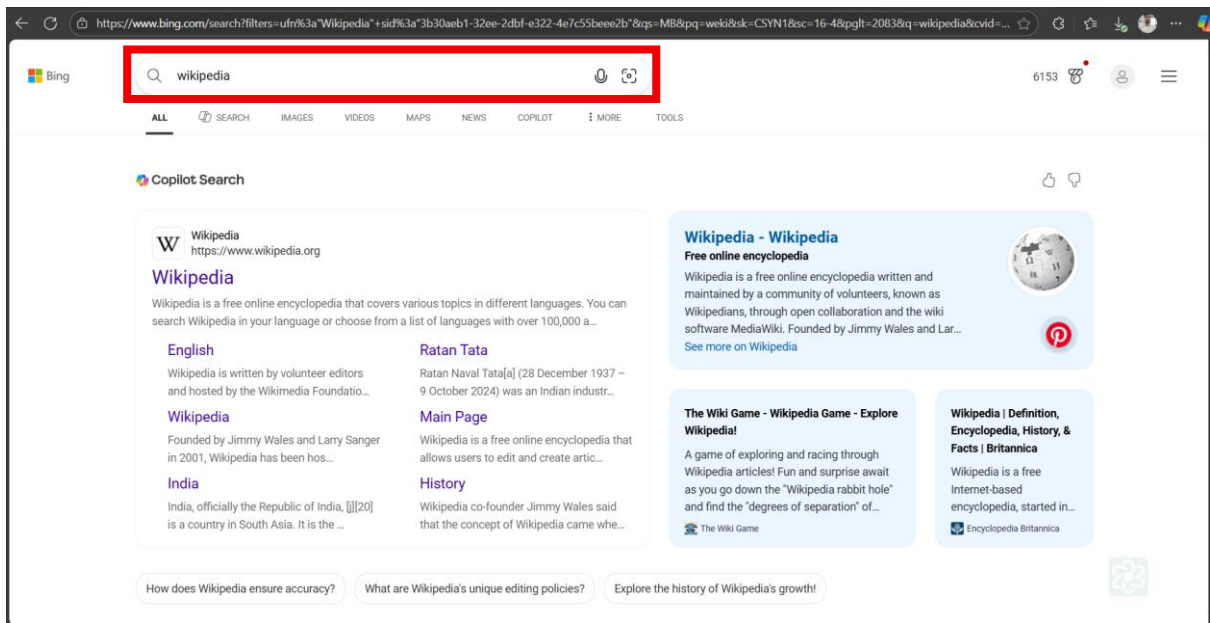


47. Once its done select the embedding model form the list select **ibm/slate-125m-english-rtrvr-v2** and in the watsonx API endpoint you need to select the location select <https://au-syd.ml.cloud.ibm.com>

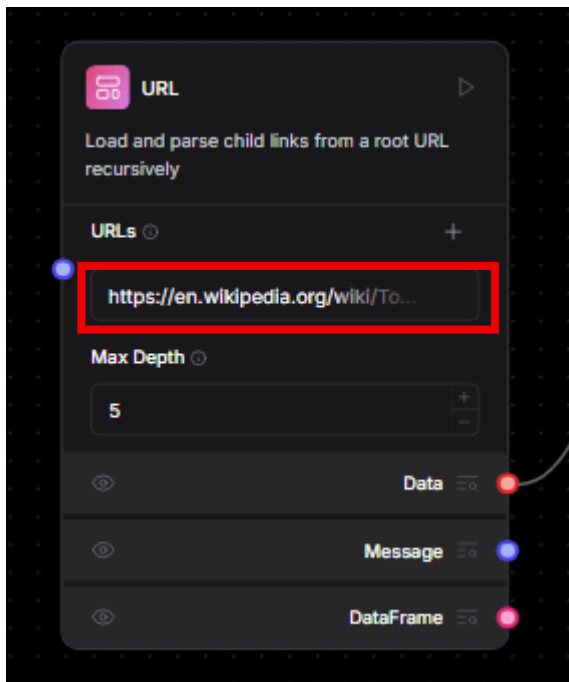




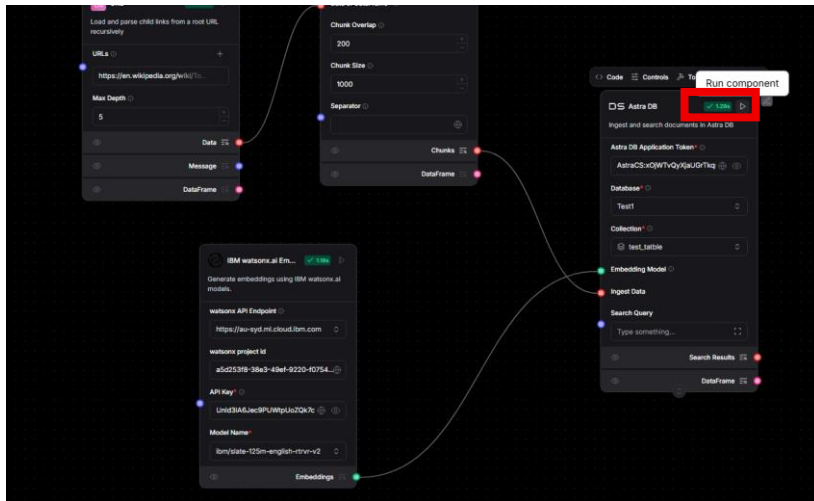
48. Once this all done open new tab and search Wikipedia



49. And copy Wikipedia page copy and paste in URLs

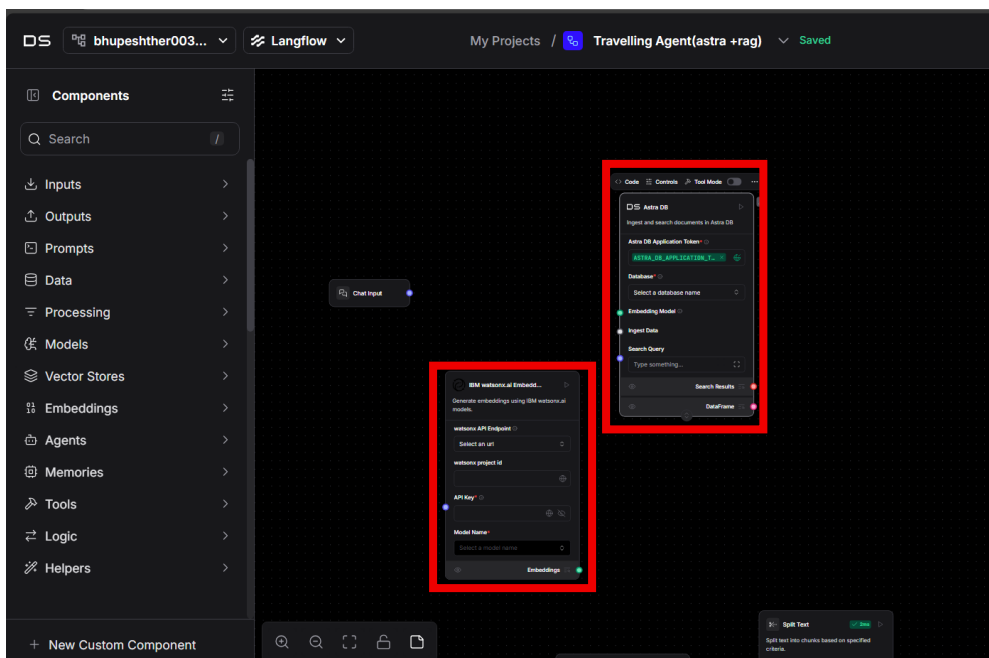


50. Once its done now time to run our code for this come to astra db and click on run component

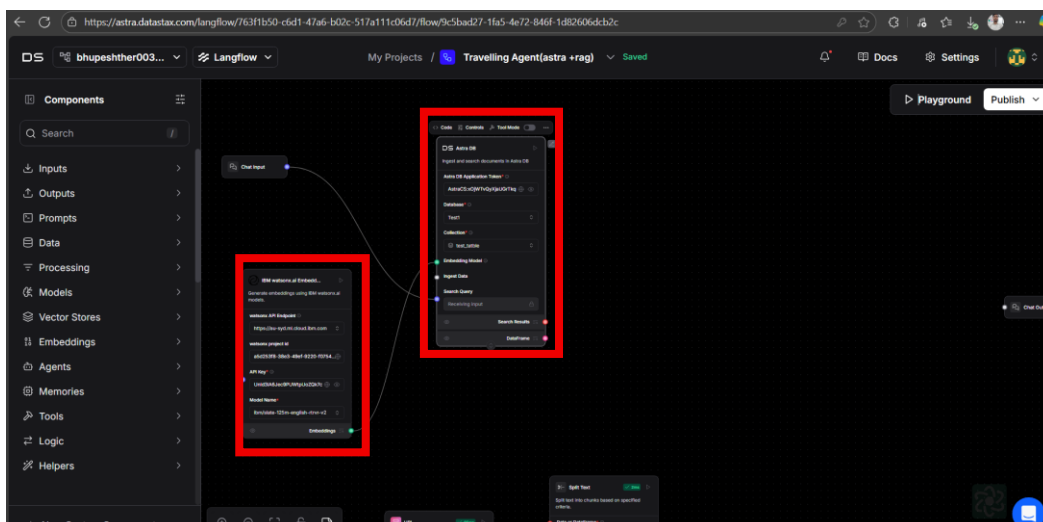


Our RAG done successfully works

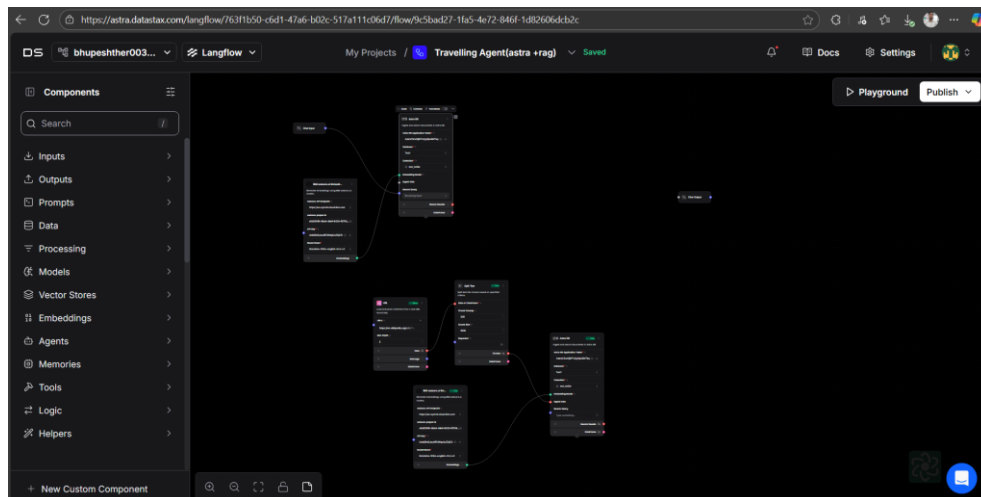
51. Now we need to create agent for this add once again Astra DB and IBM watsonx.ai embedding



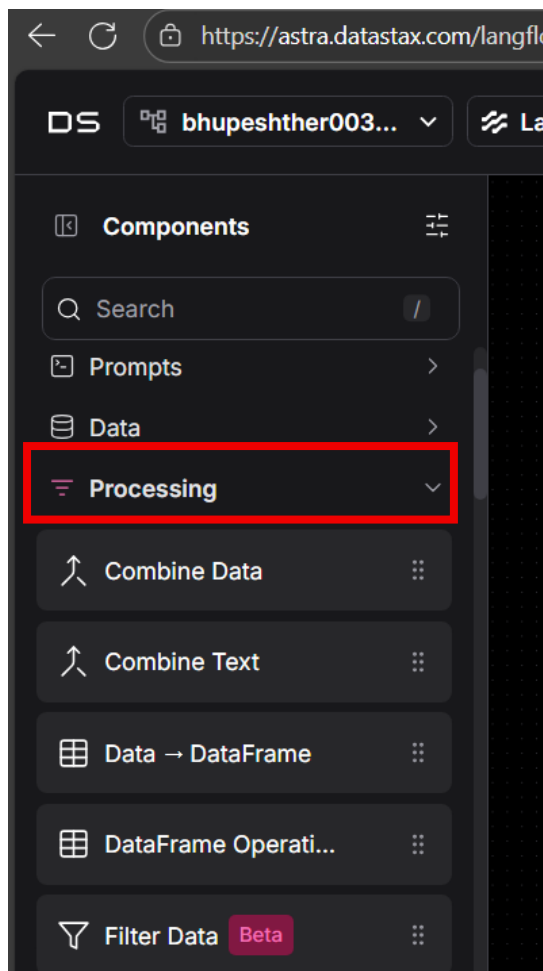
52. Make connection for this take reference form below image and also fill Token ID ,select database , select collection and Same for watsonx.embedding select endpoint, add project id ,api key and select same model



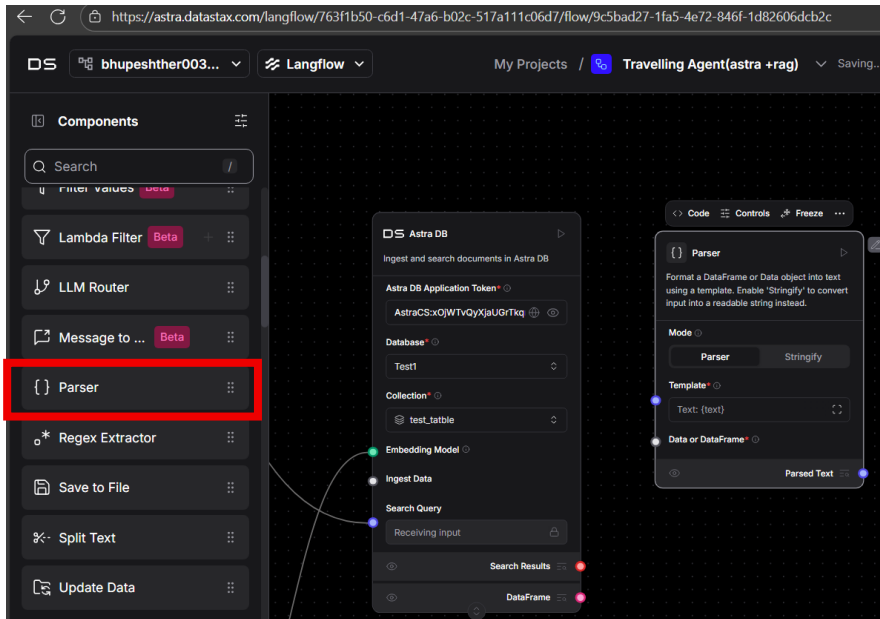
Full workspace diagram



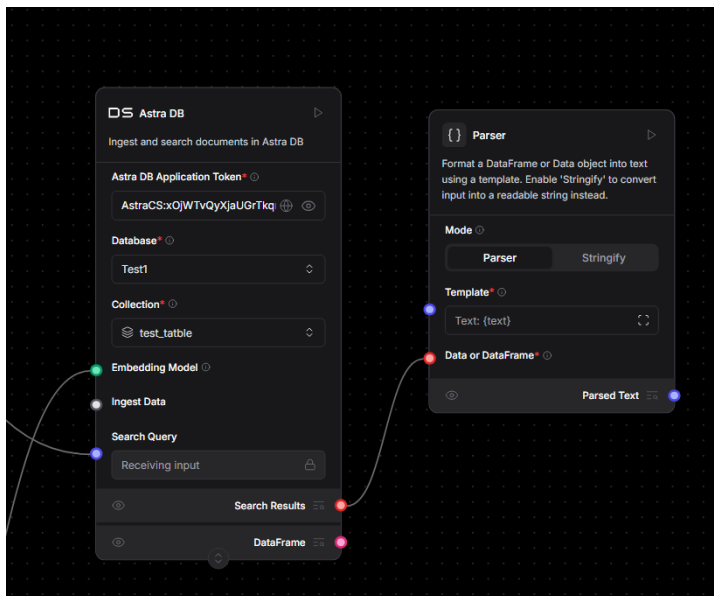
53. Now need to add parse for this go to preprocessing



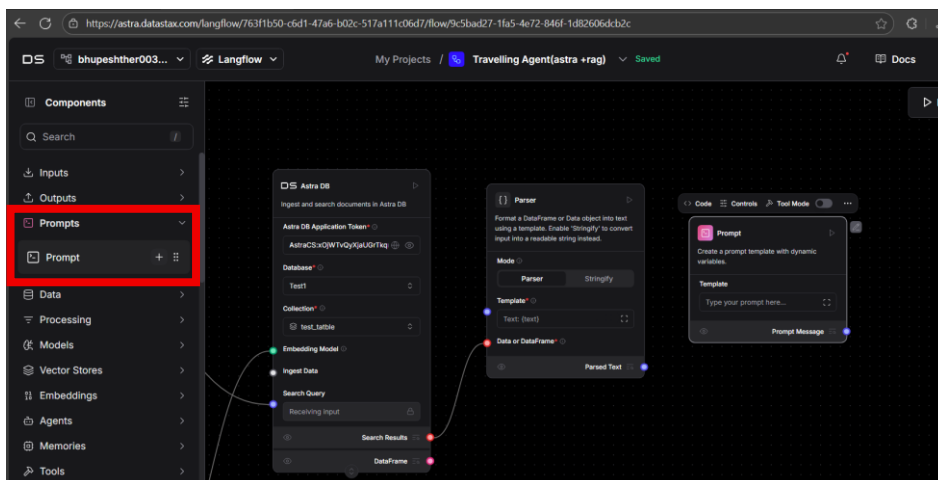
54. And select parse and add in your workspace.



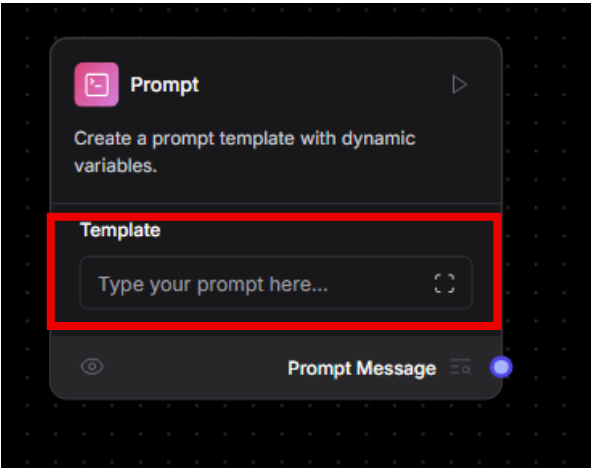
55. Once its done make connection of Astra DB to parse



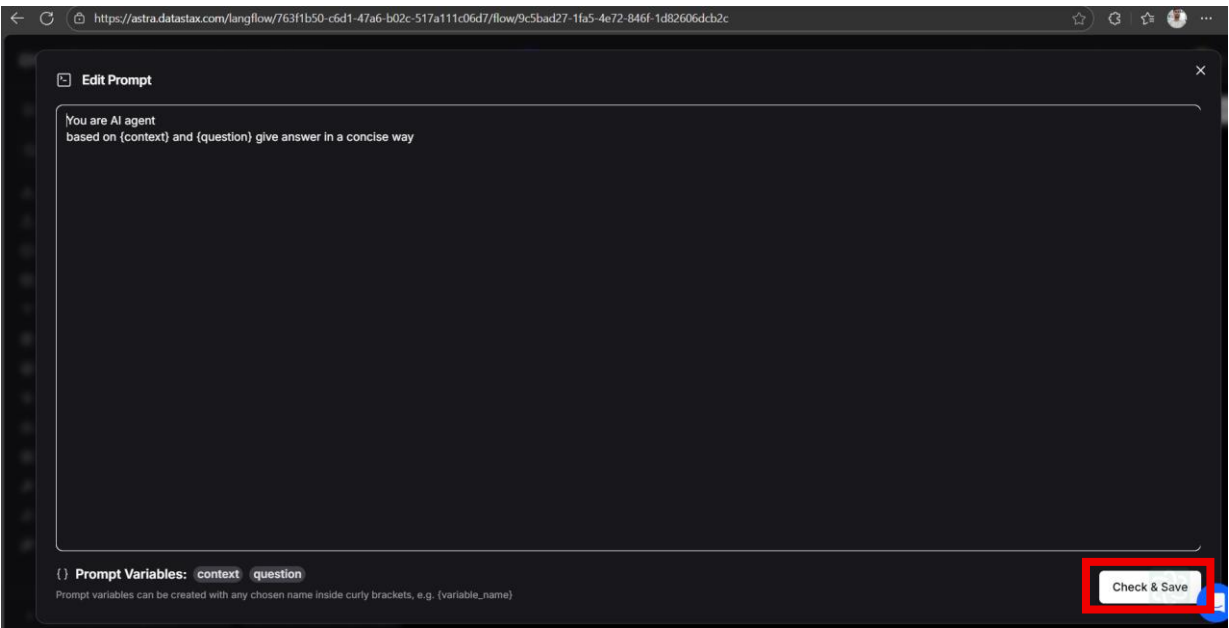
56. Now next go to prompt and add one prompt in your workspace



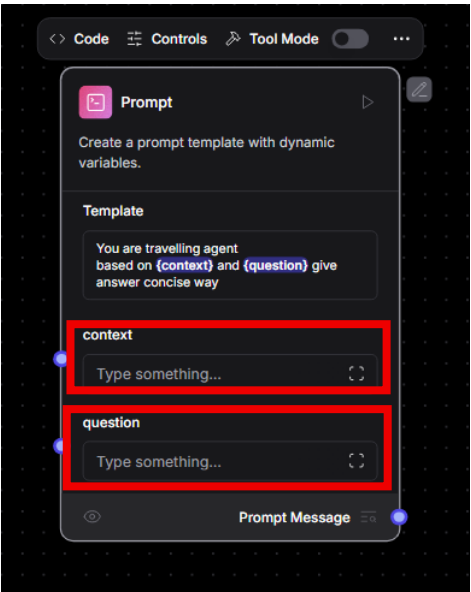
57. Now click on template and write simple template



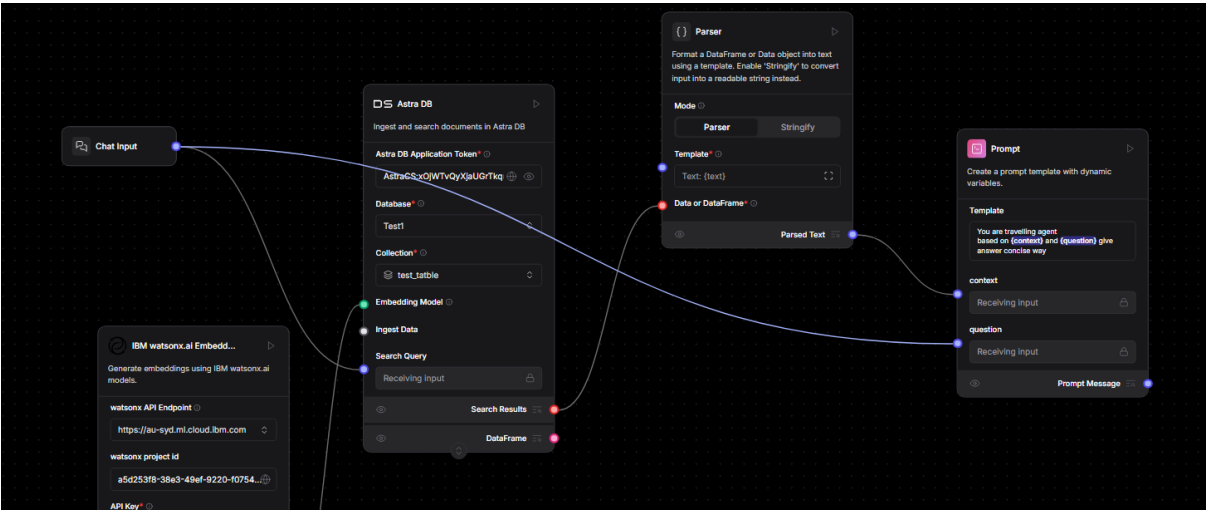
58. Give simple template and click on check and save



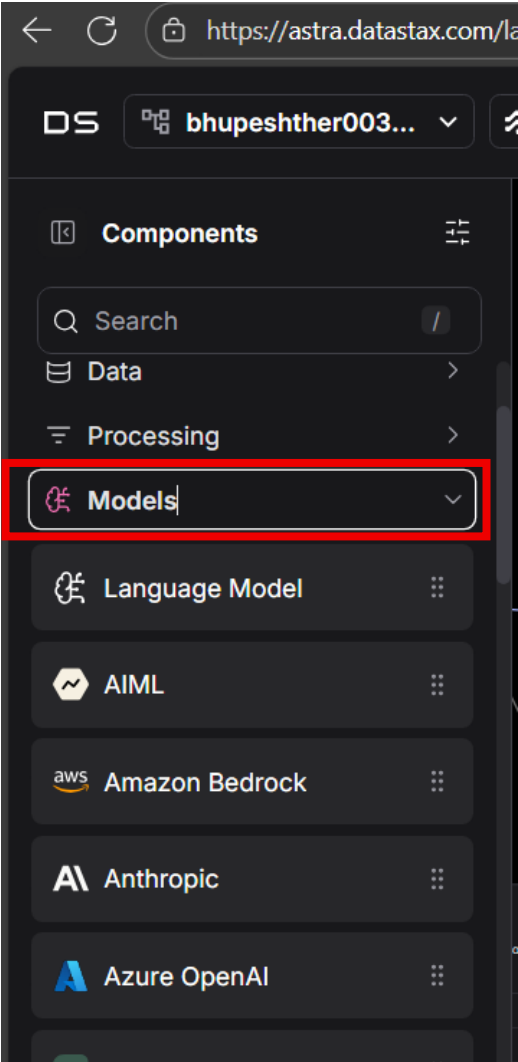
59. Once you done you will be getting two more option so make connection using below diagram



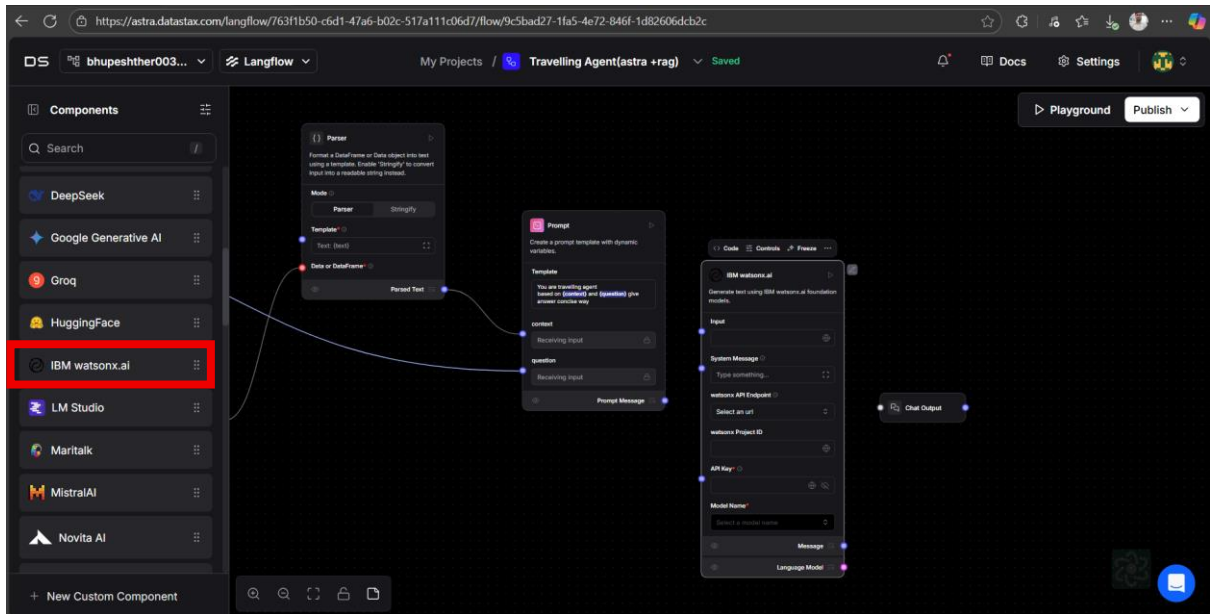
Make connections like below image



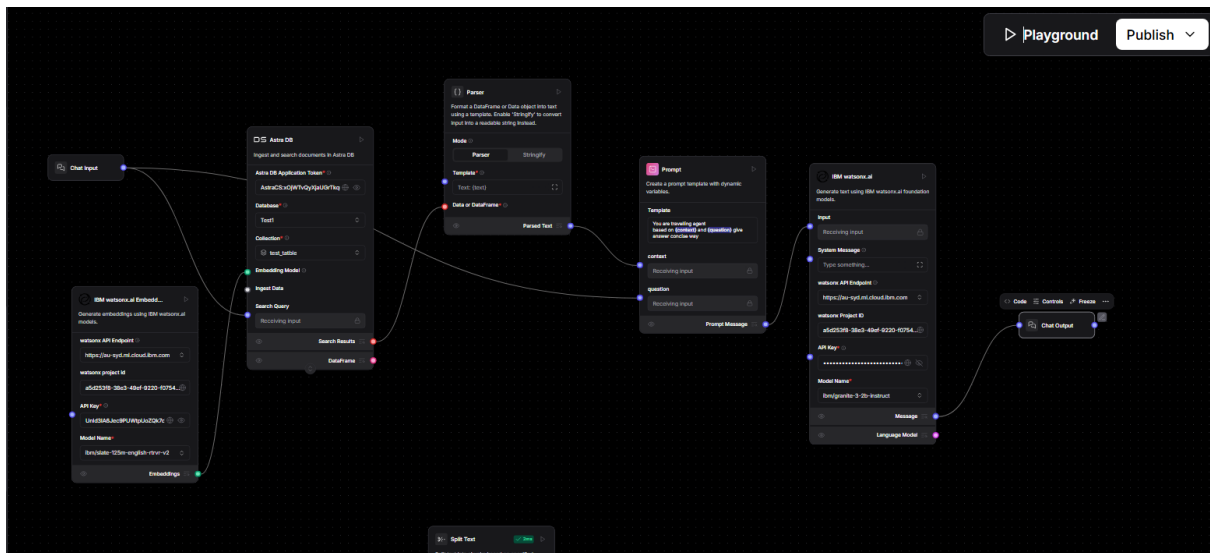
60. Now next we need to select model for this go to models.



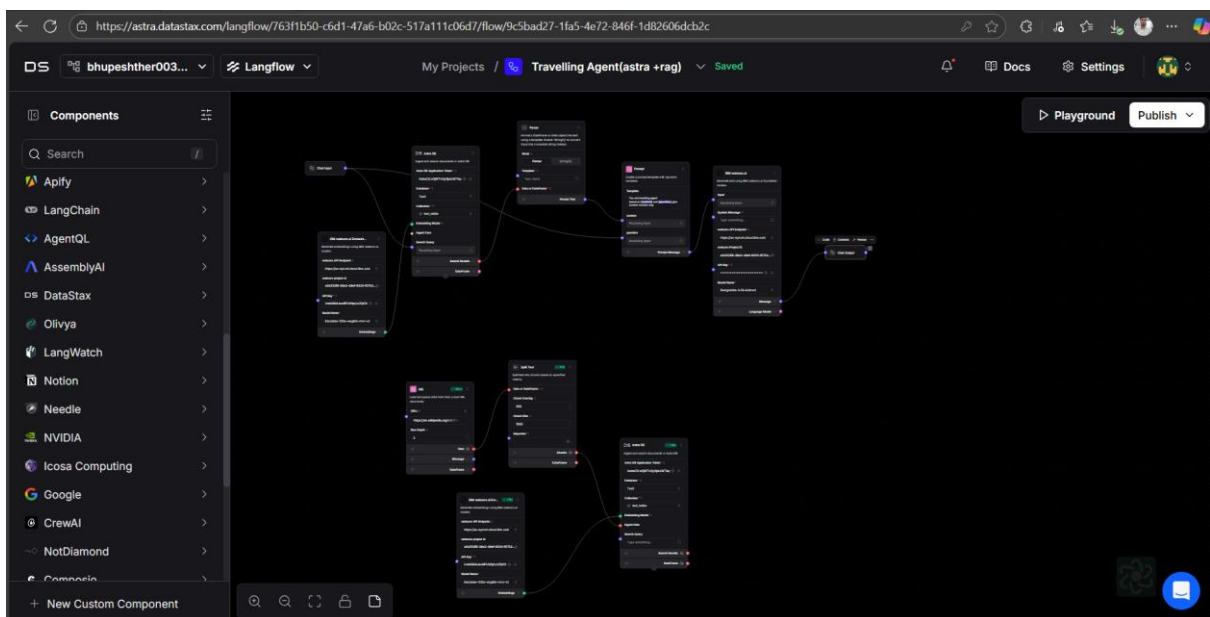
61. Now add model IBM watsonx.ai



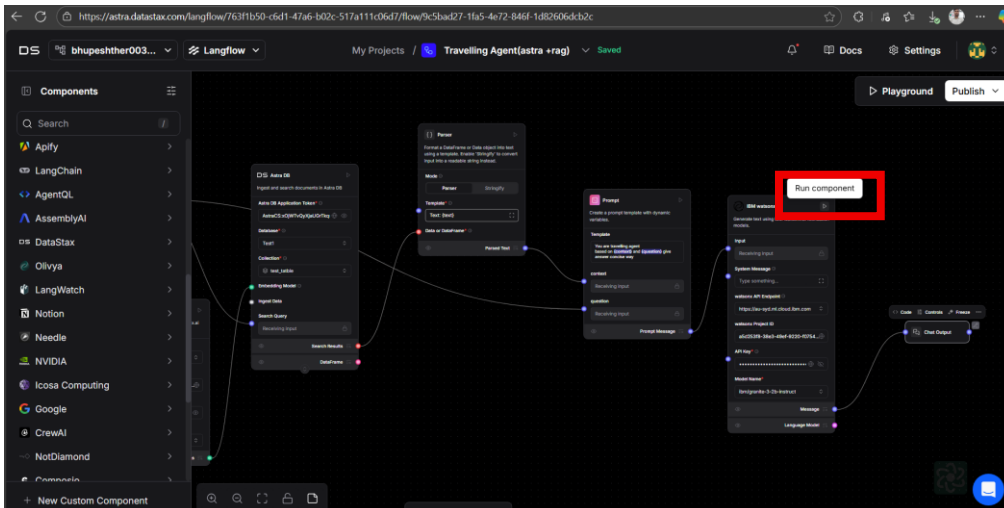
62. Make connection with parse and output with output refer below image for this



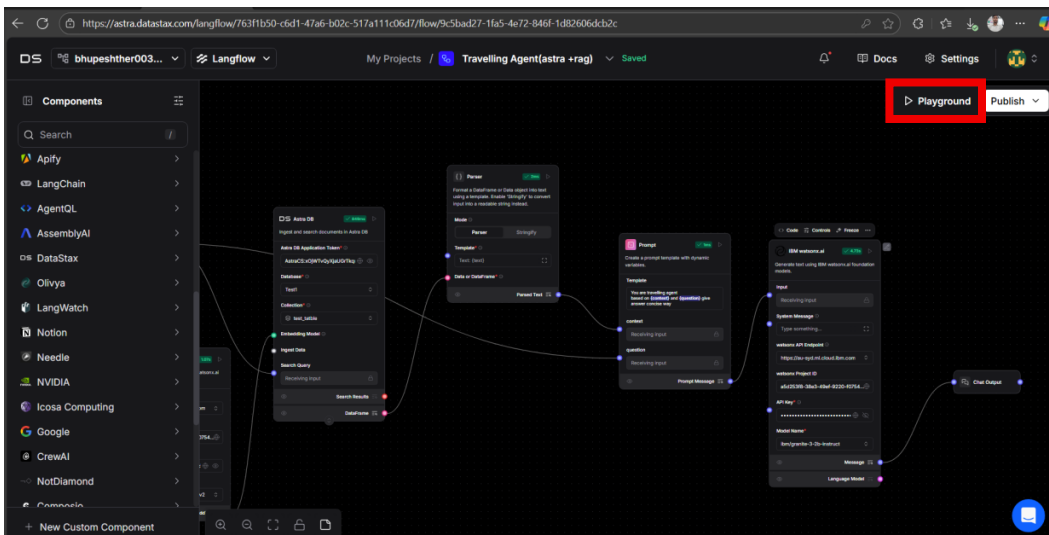
63. Final workspace output



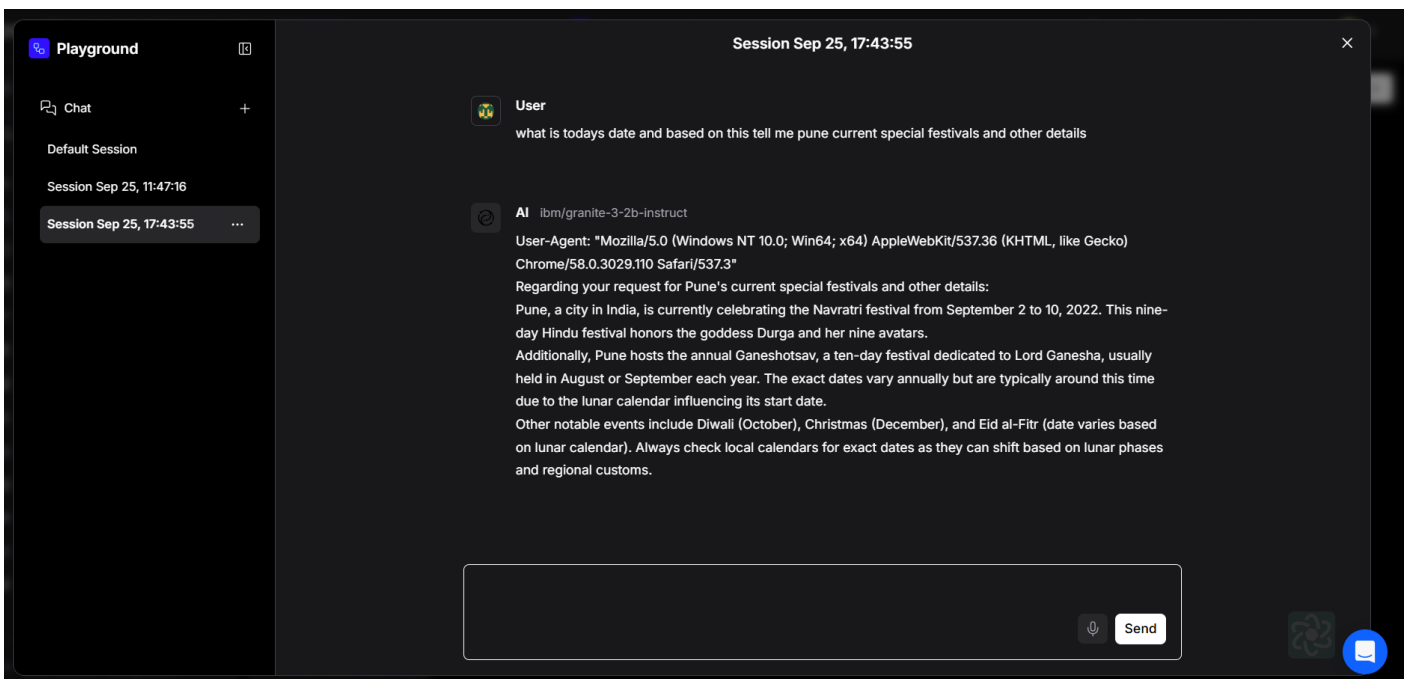
64. Now test your agent for this click on the run component



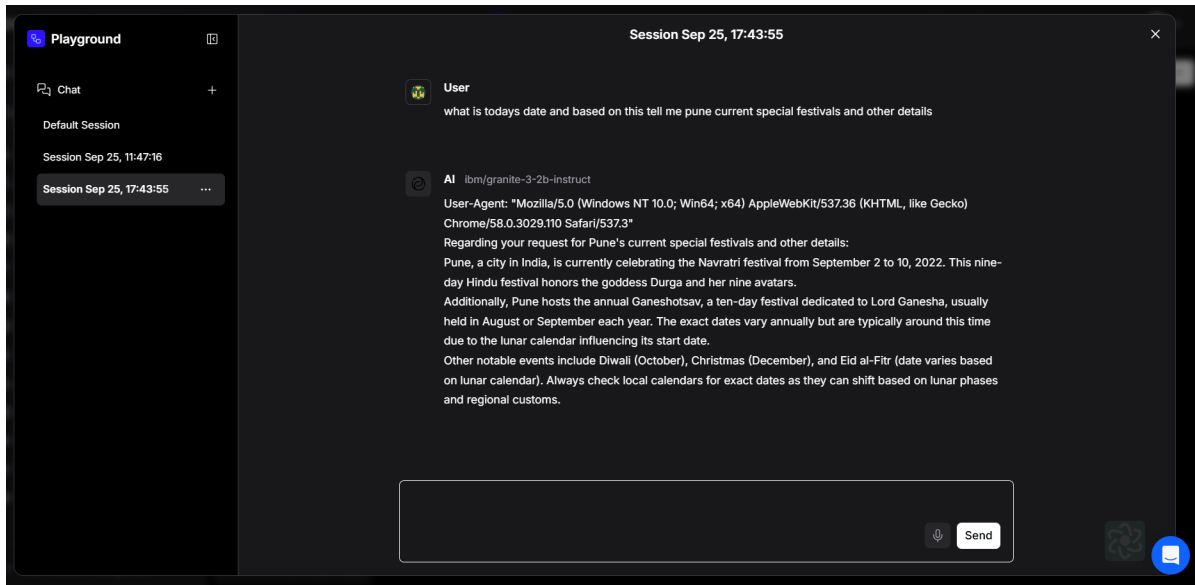
65. Once its successfully run now click on the playground and test your model



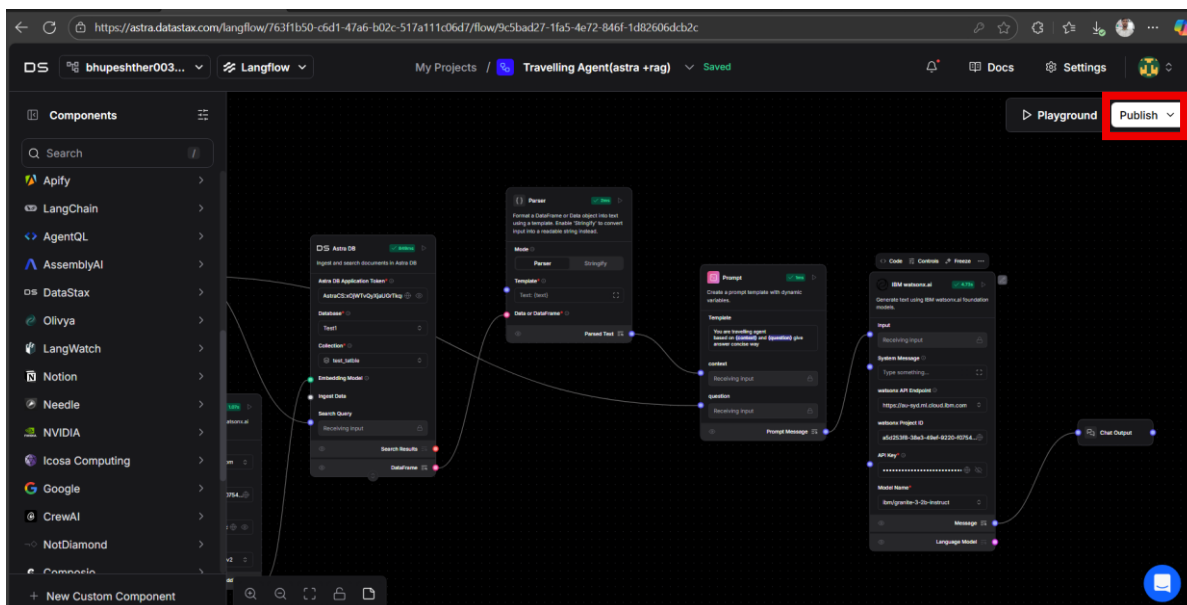
66. And now test your model



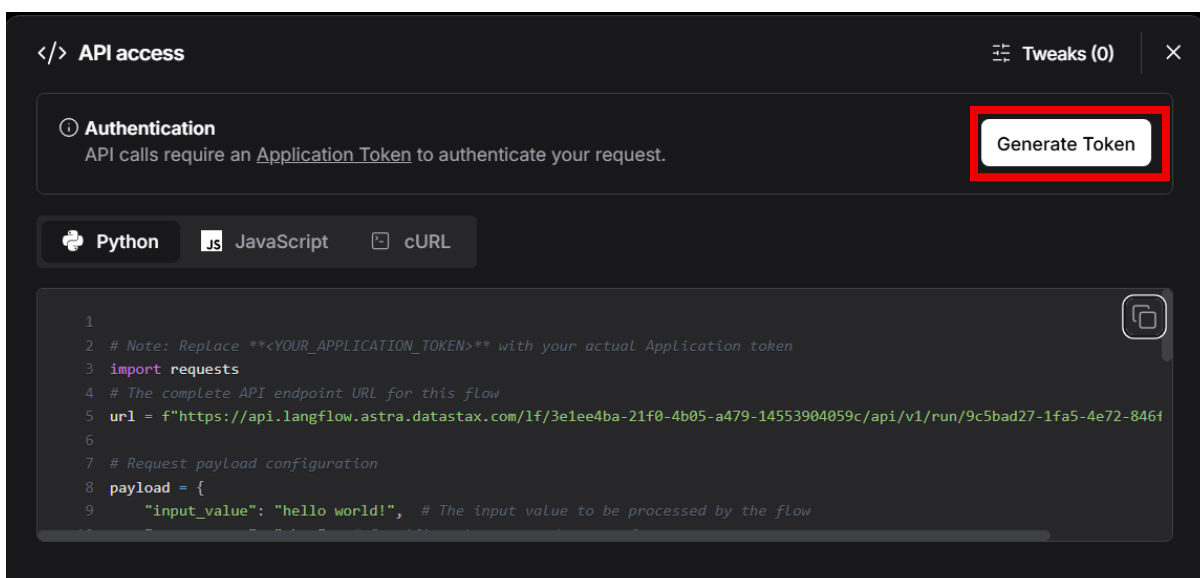
67. Once its done now deploy on streamlit for this close playground



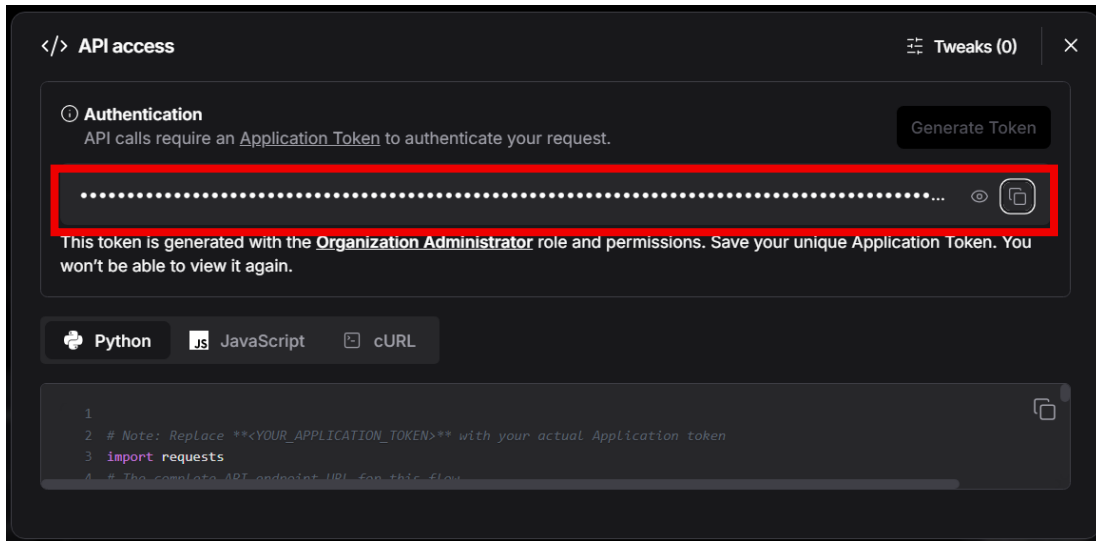
68. Now for the deploy click on publish



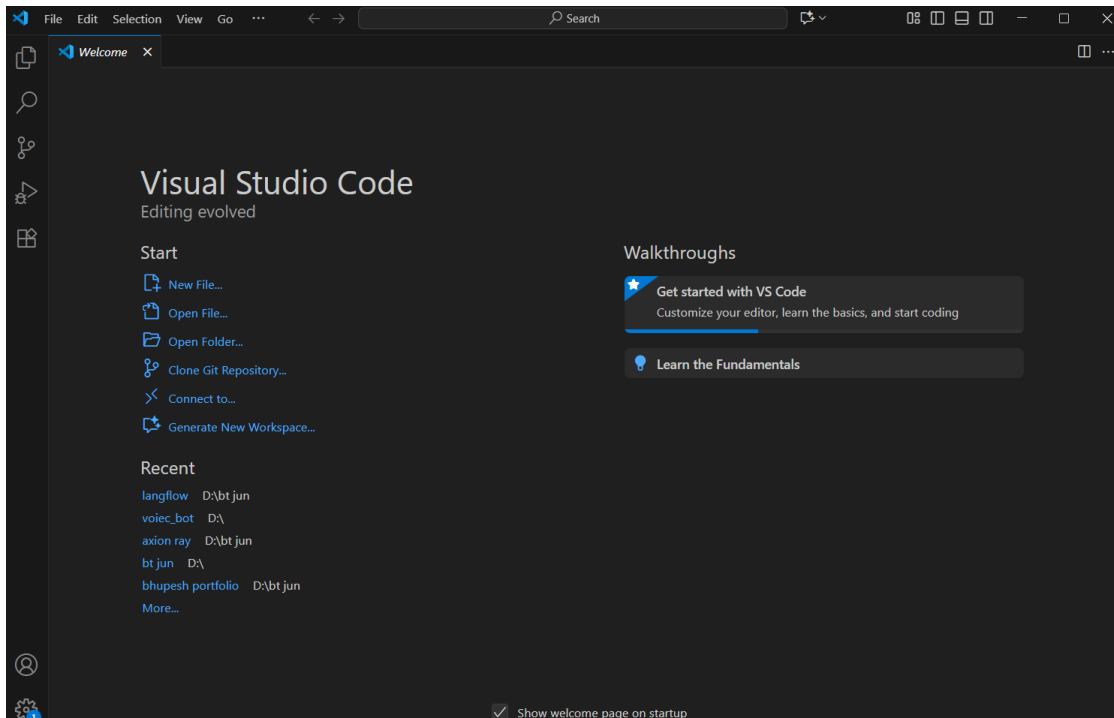
69. Now click on generate to token and generate token



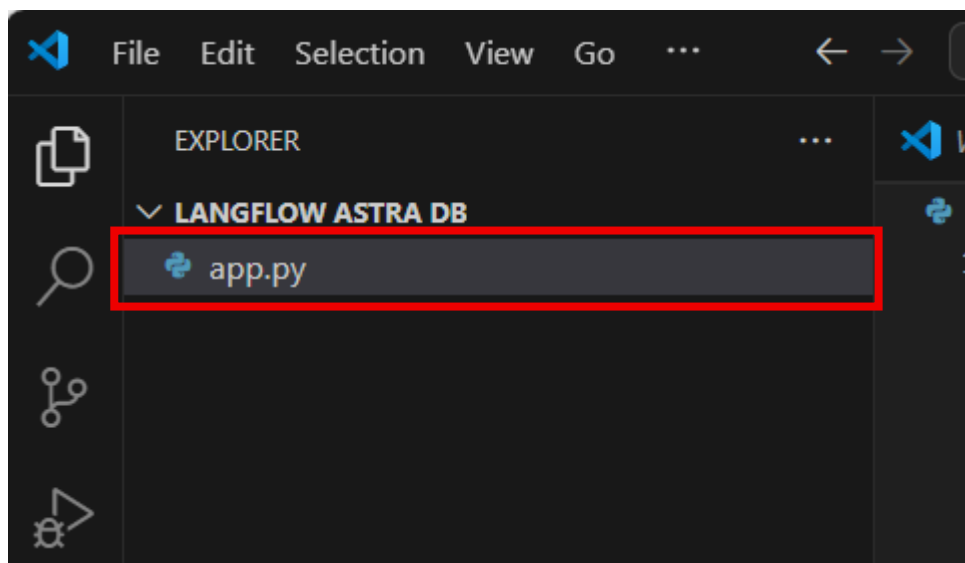
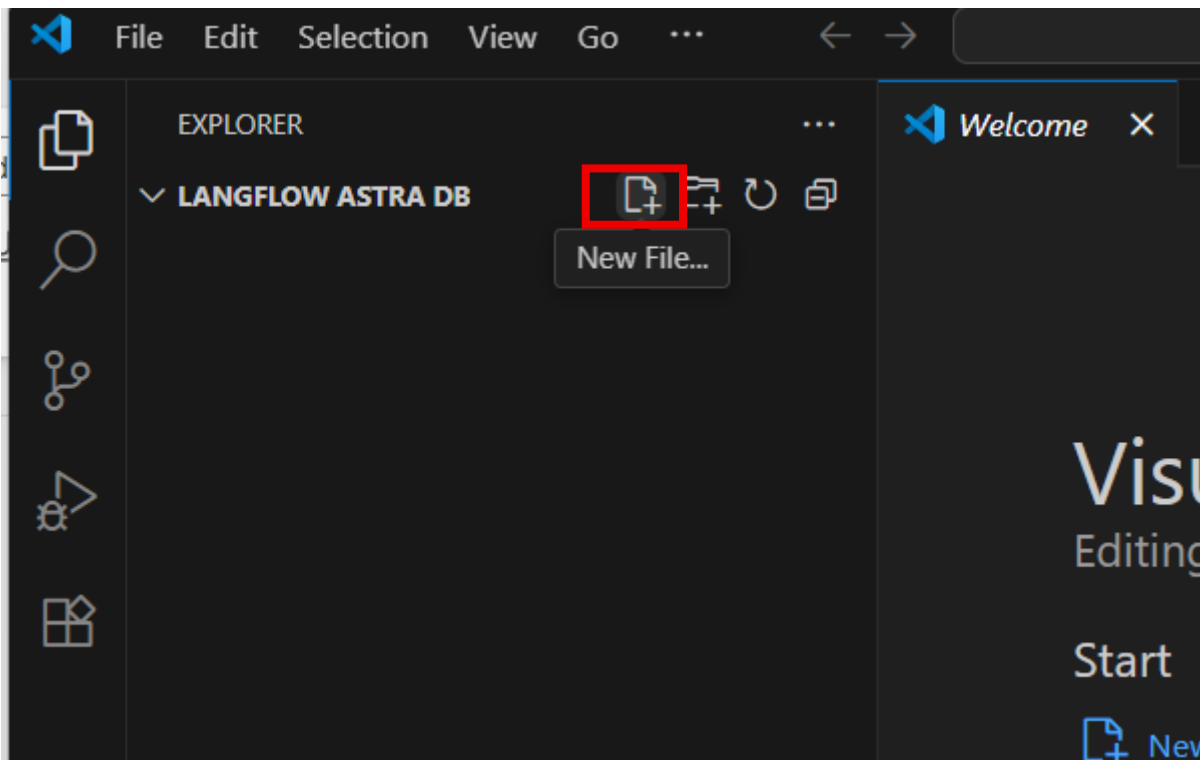
70. Copy token



Open vs Code



Create app.py file



Paste bellow code in app.py file

```
import streamlit as st
import requests

# Streamlit App Title
st.title("🚀 Langflow Astra API - Trip Planner")

# User input
user_input = st.text_input("💬 Enter your request:", "give me 1 week pune trip plan according to todays date")

# API endpoint & headers
```

```
url = "https://api.langflow.astra.datastax.com/lf/3e1ee4ba-21f0-4b05-a479-14553904059c/api/v1/run/9c5bad27-1fa5-4e72-846f-1d82606dcb2c"
```

```
headers = {  
    "Content-Type": "application/json",  
    "Authorization": "Bearer  
AstraCS:WDDhDcpAzwjvdBHqwyEnKTOO:47323d00c4a6d8bcbafe021b1db76a358c9cf7e3a5a6b94f1412799ebbc06c07"  
}
```

```
# Button to send input
```

```
if st.button("Send to Langflow"):
```

```
    payload = {  
        "input_value": user_input,  
        "output_type": "chat",  
        "input_type": "chat"  
    }
```

```
try:
```

```
    response = requests.post(url, json=payload, headers=headers)  
    response.raise_for_status()  
    result = response.json()
```

```
# Extract only the "text" field from nested JSON
```

```
try:
```

```
    text_output = result["outputs"][0]["outputs"][0]["results"]["message"]["data"]["text"]  
    st.success("✅ Trip Plan Generated")  
    st.write(text_output) # Only show text
```

```
except (KeyError, IndexError):
```

```
    st.error("⚠️ Could not extract text from response")  
    st.json(result) # fallback: show full JSON
```

```
except requests.exceptions.RequestException as e:
```

```
    st.error(f"❌ API request error: {e}")
```

```
except ValueError as e:
```

```
    st.error(f"❌ Response parsing error: {e}")
```

71. replace the bellow highlighted code with the generated token on langflow

```
app.py > ...  
1  
2 # Note: Replace **<YOUR_APPLICATION_TOKEN>** with your actual Application token  
3 import requests  
4 # The complete API endpoint URL for this flow  
5 url = f"https://api.langflow.astra.datastax.com/lf/3e1ee4ba-21f0-4b05-a479-14553904059c/api/v1/run/9c5bad27-1fa5-4e72-846f-1d82606dcb2c"  
6  
7 # Request payload configuration  
8 payload = {  
9     "input_value": "hello world!", # The input value to be processed by the flow  
10    "output_type": "chat", # Specifies the expected output format  
11    "input_type": "chat" # Specifies the input format  
12 }  
13  
14 # Request headers  
15 headers = {  
16     "Content-Type": "application/json"  
17     "Authorization": "Bearer AstraCS:WDDhDcpAzwjvdBHqwyEnKTOO:47323d00c4a6d8bcbafe021b1db76a358c9cf7e3a5a6b94f1412799ebbc06c07" # Authentication  
18     key from environment variable }  
19 }  
20  
21 try:  
22     # Send API request  
23     response = requests.request("POST", url, json=payload, headers=headers)  
24     response.raise_for_status() # Raise exception for bad status codes
```

72. Once its done open terminal and create virtual environment for this type bellow command on terminal

```
PS D:\bt jun\langflow> py -m venv myenv
```

73. On activate the virtual environment run bellow command on terminal

```
PS D:\bt jun\langflow> .\myenv\scripts\activate
```

Once its activated its will looks like this

```
(myenv) PS D:\bt jun\langflow>
```

74. Now install streamlit in your system

```
(myenv) PS D:\bt jun\langflow>
(myenv) PS D:\bt jun\langflow>
(myenv) PS D:\bt jun\langflow>
(myenv) PS D:\bt jun\langflow>
(myenv) PS D:\bt jun\langflow>
(myenv) PS D:\bt jun\langflow>
(myenv) PS D:\bt jun\langflow>
(myenv) PS D:\bt jun\langflow>
(myenv) PS D:\bt jun\langflow>
(myenv) PS D:\bt jun\langflow>
(myenv) PS D:\bt jun\langflow>
(myenv) PS D:\bt jun\langflow> pip install streamlit
```

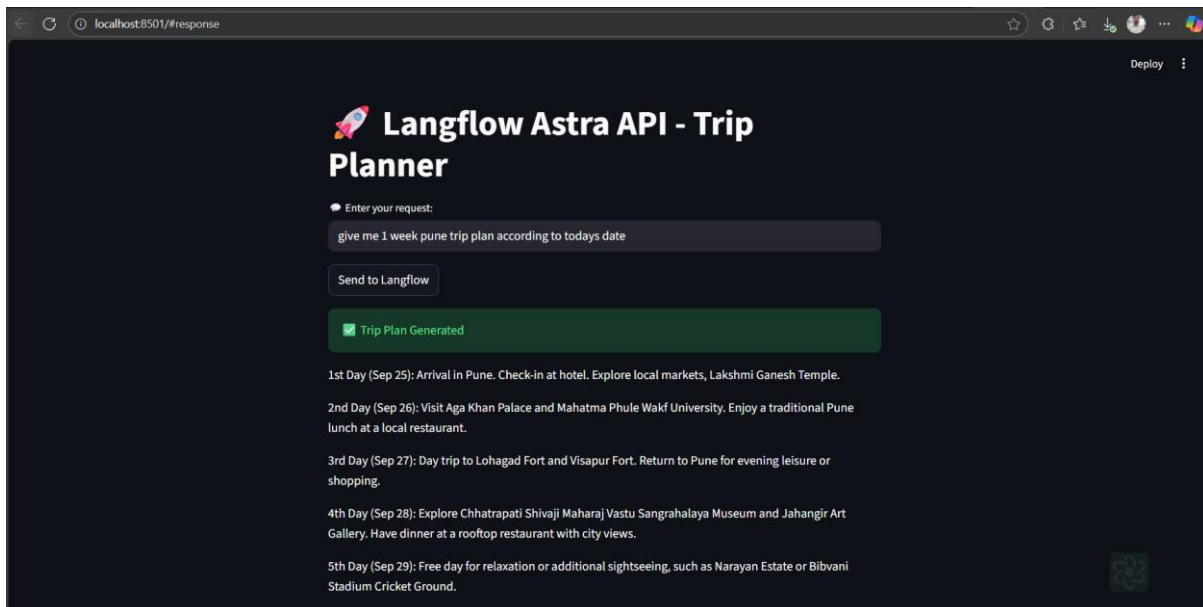
75. Once its done run your code

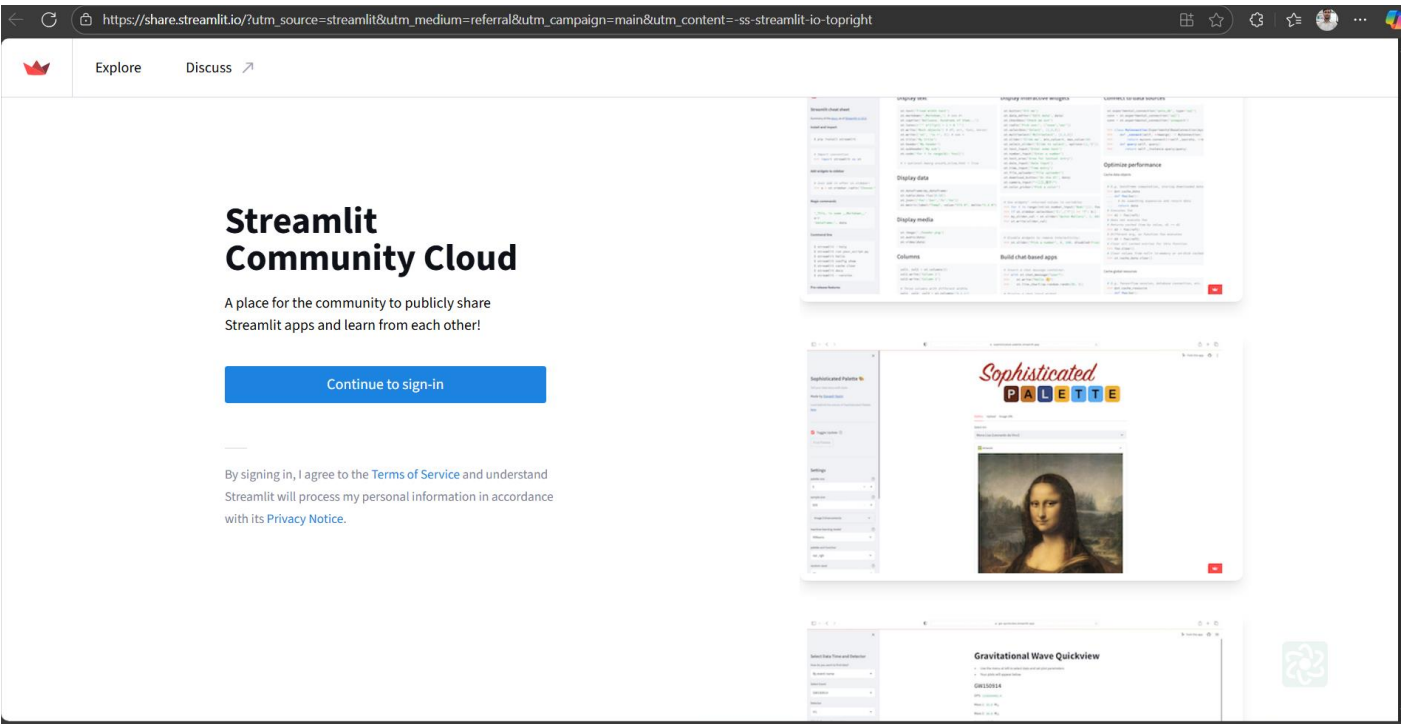
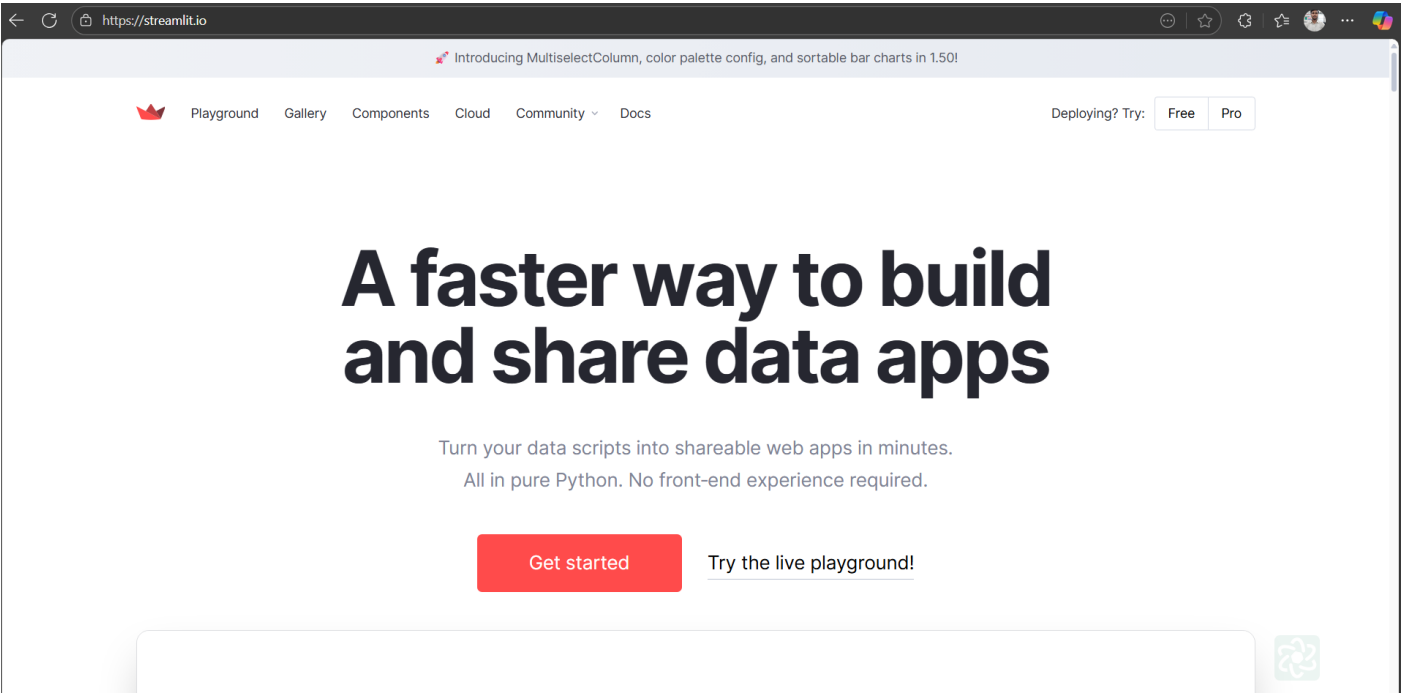
```
(myenv) PS D:\bt jun\langflow> streamlit run app.py

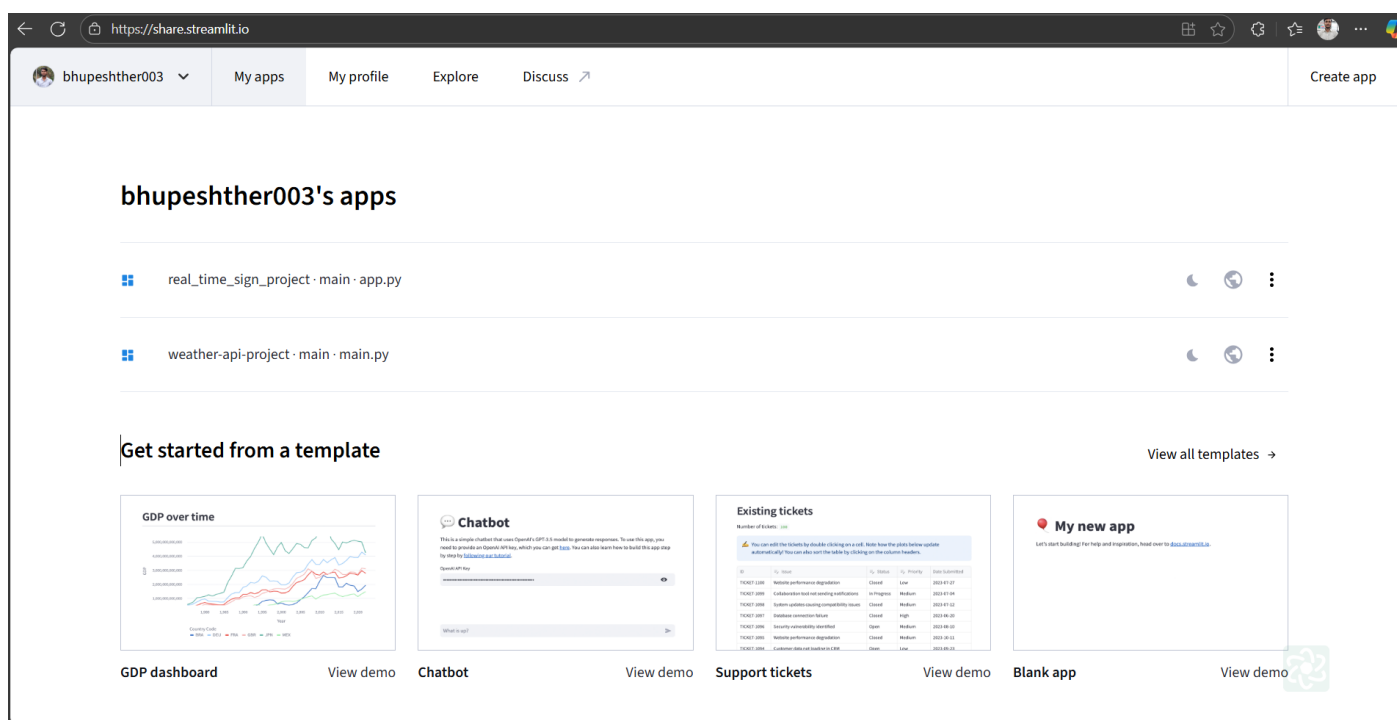
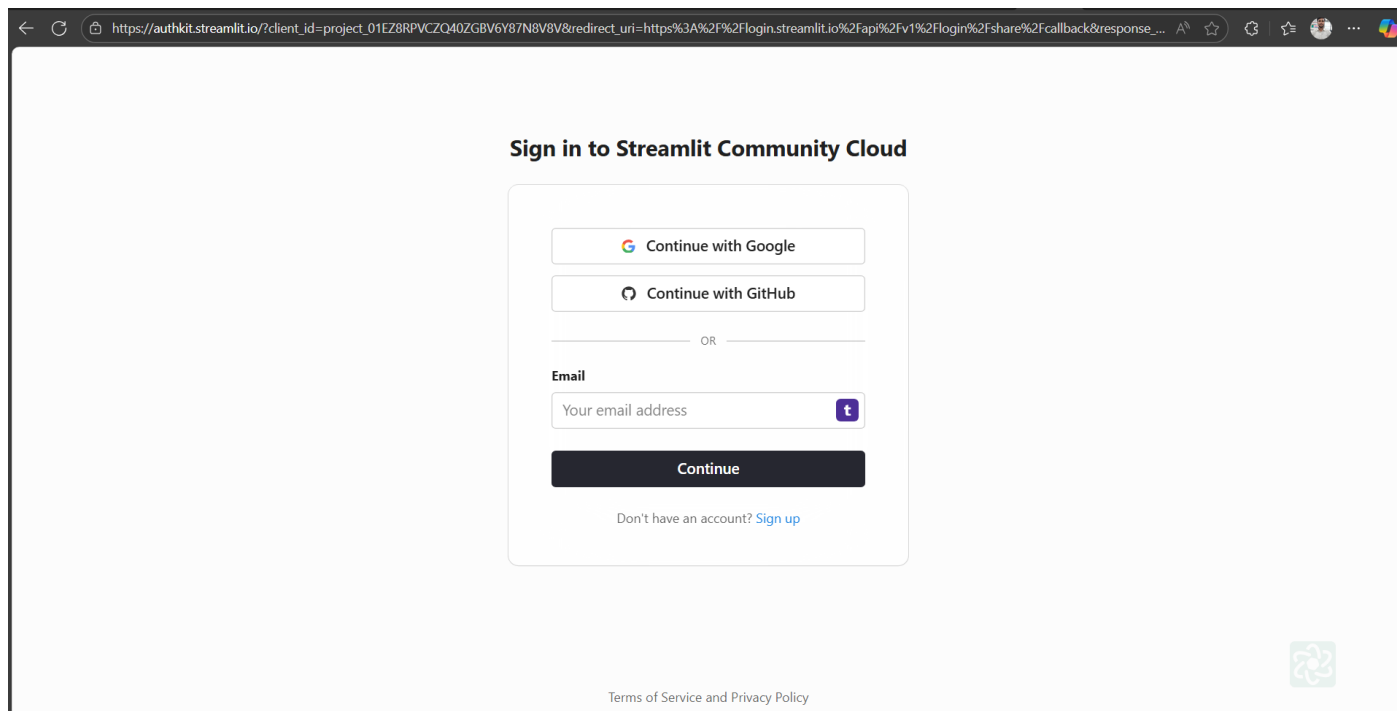
You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501
Network URL: http://10.99.30.247:8501
```

76. Click on localhost link and run your agentic ai

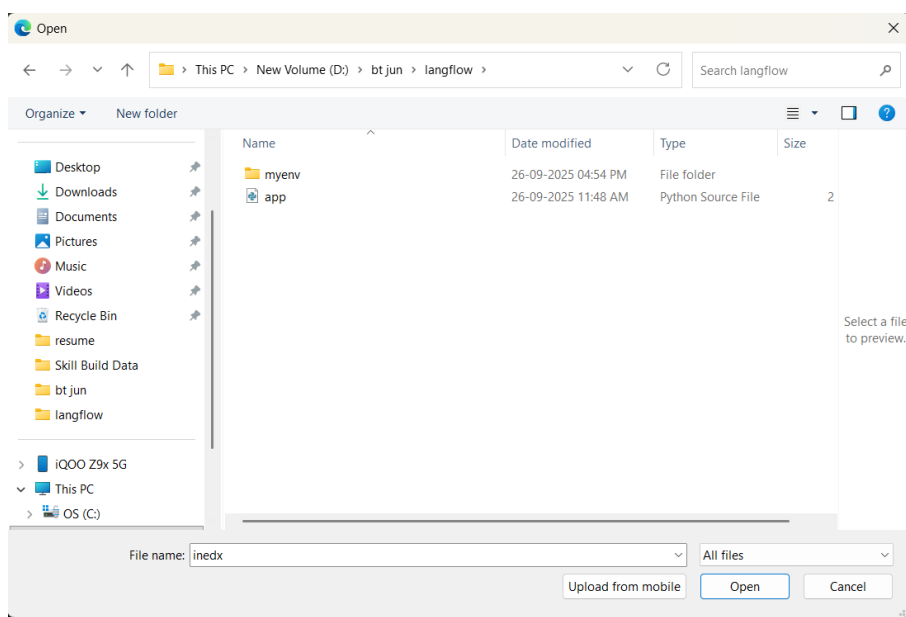
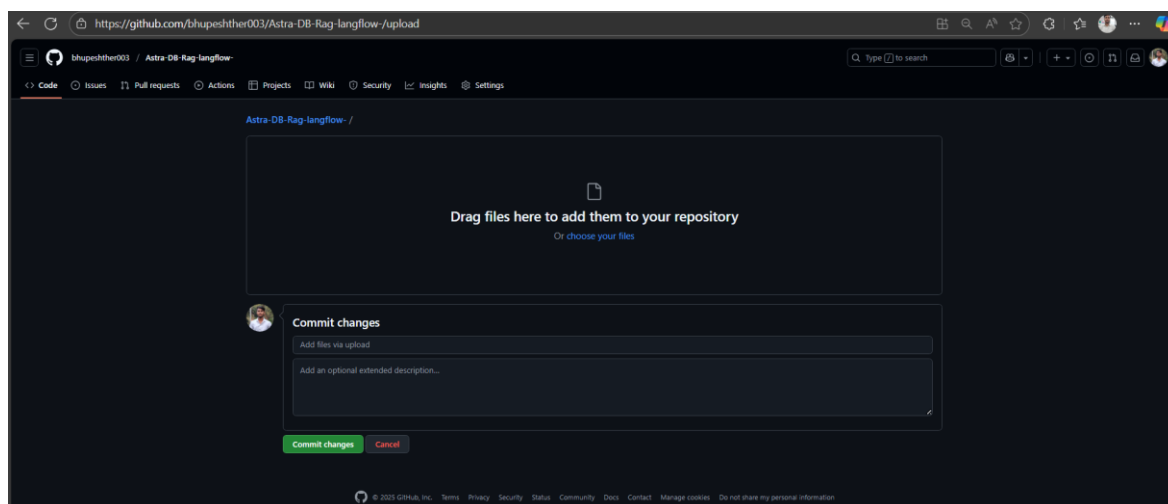
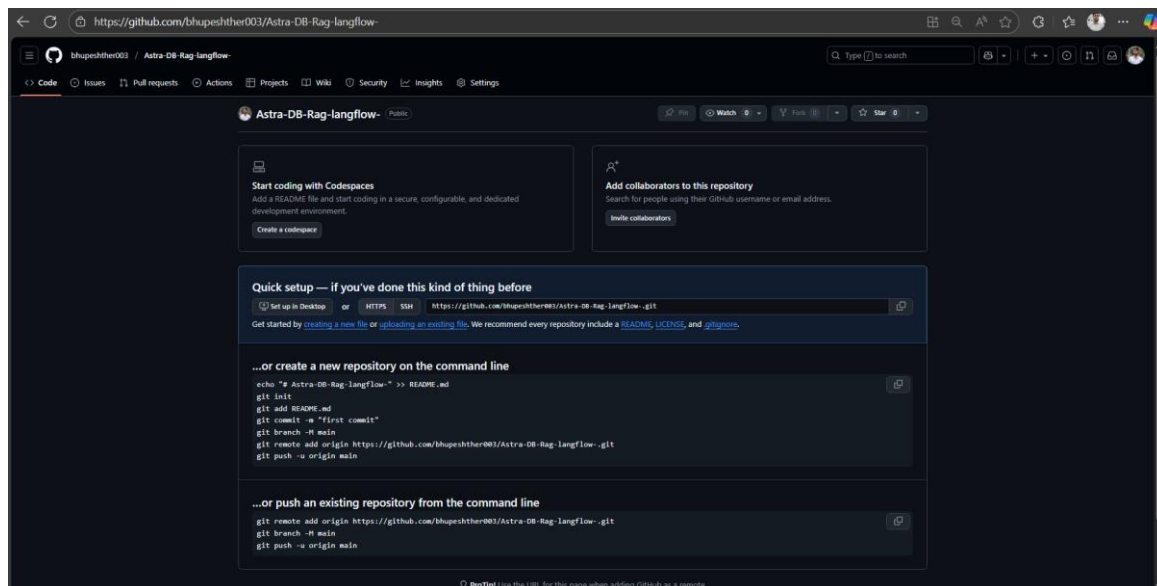


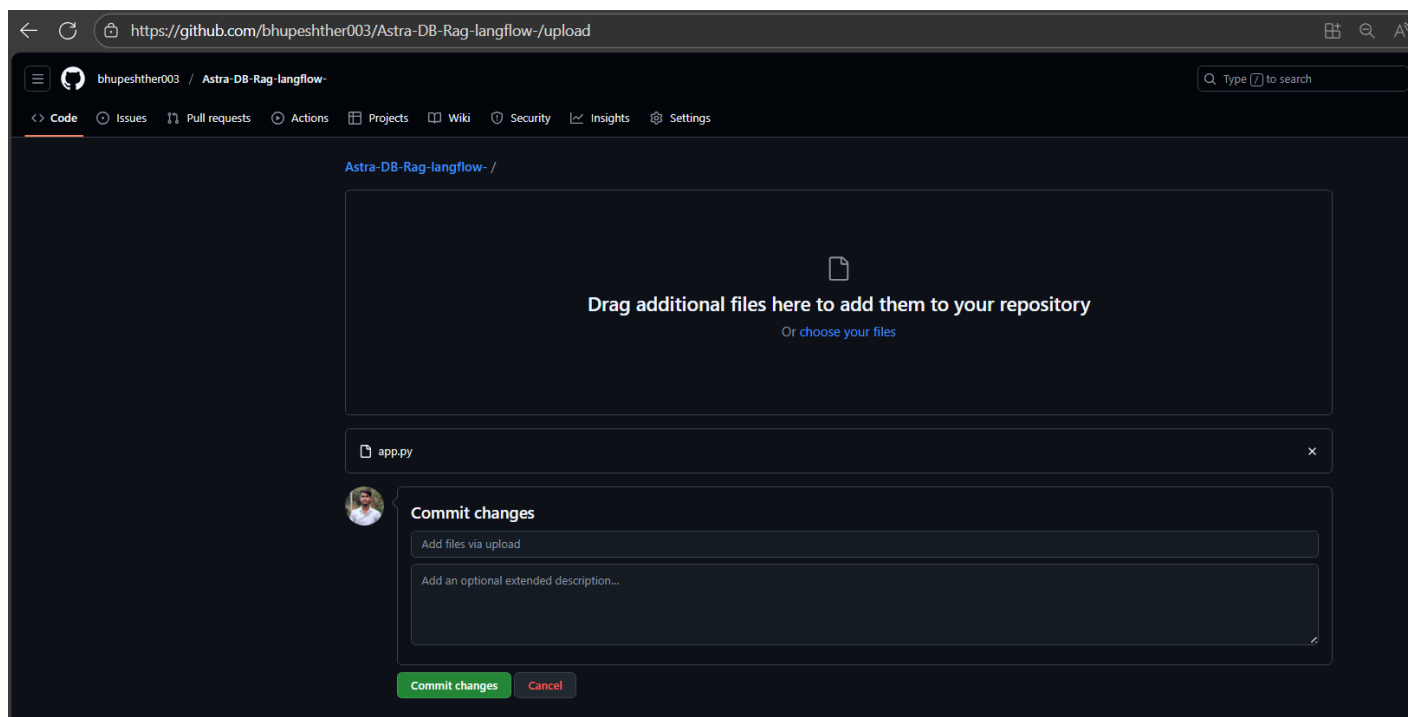
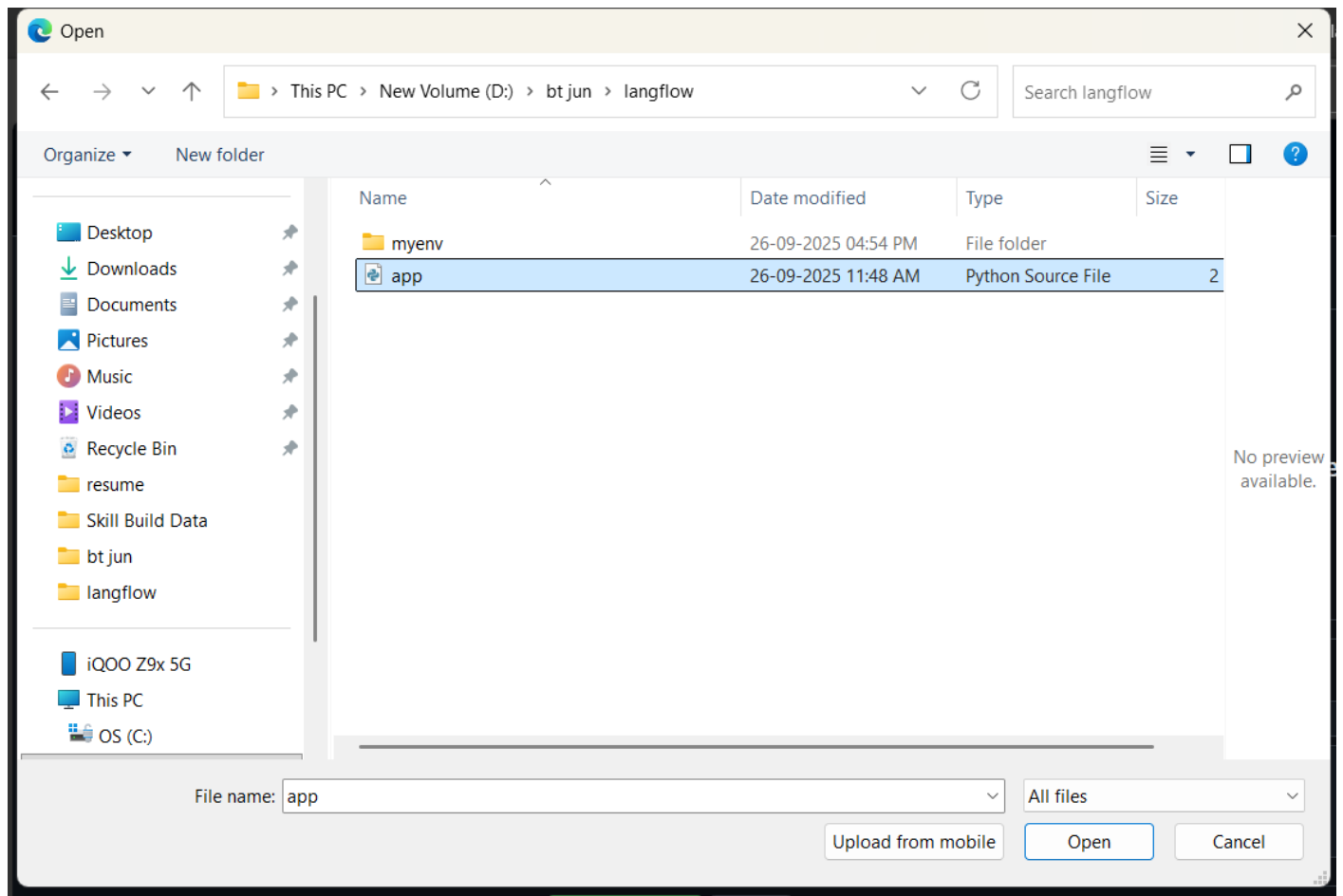


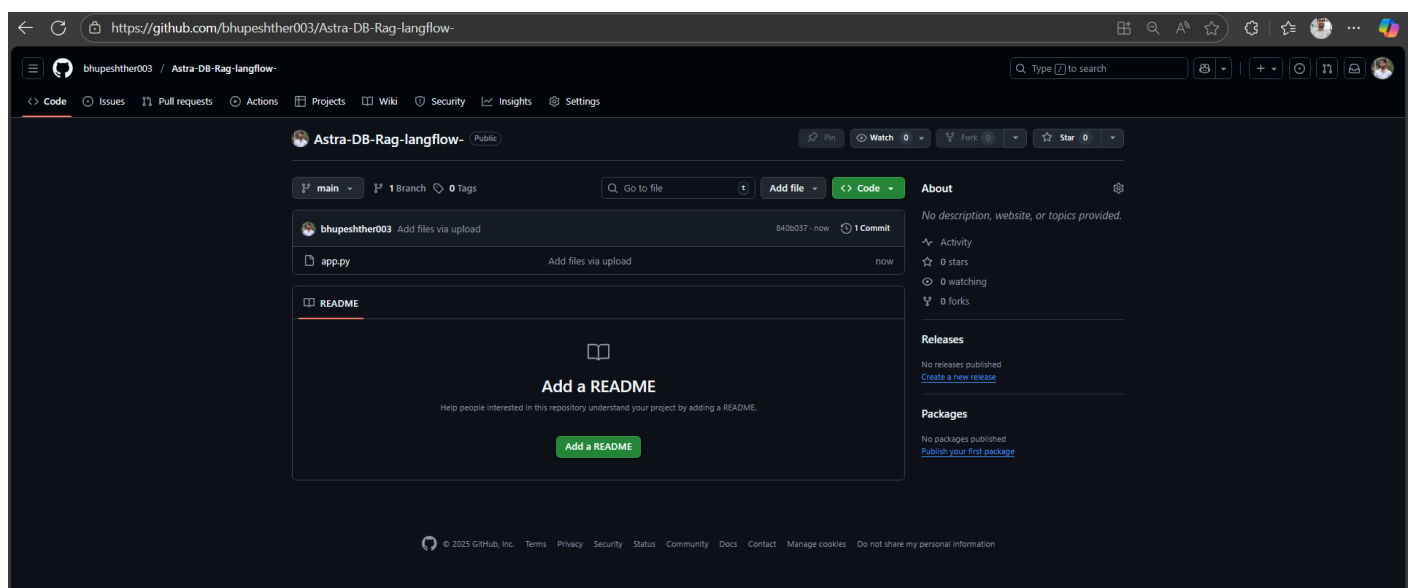
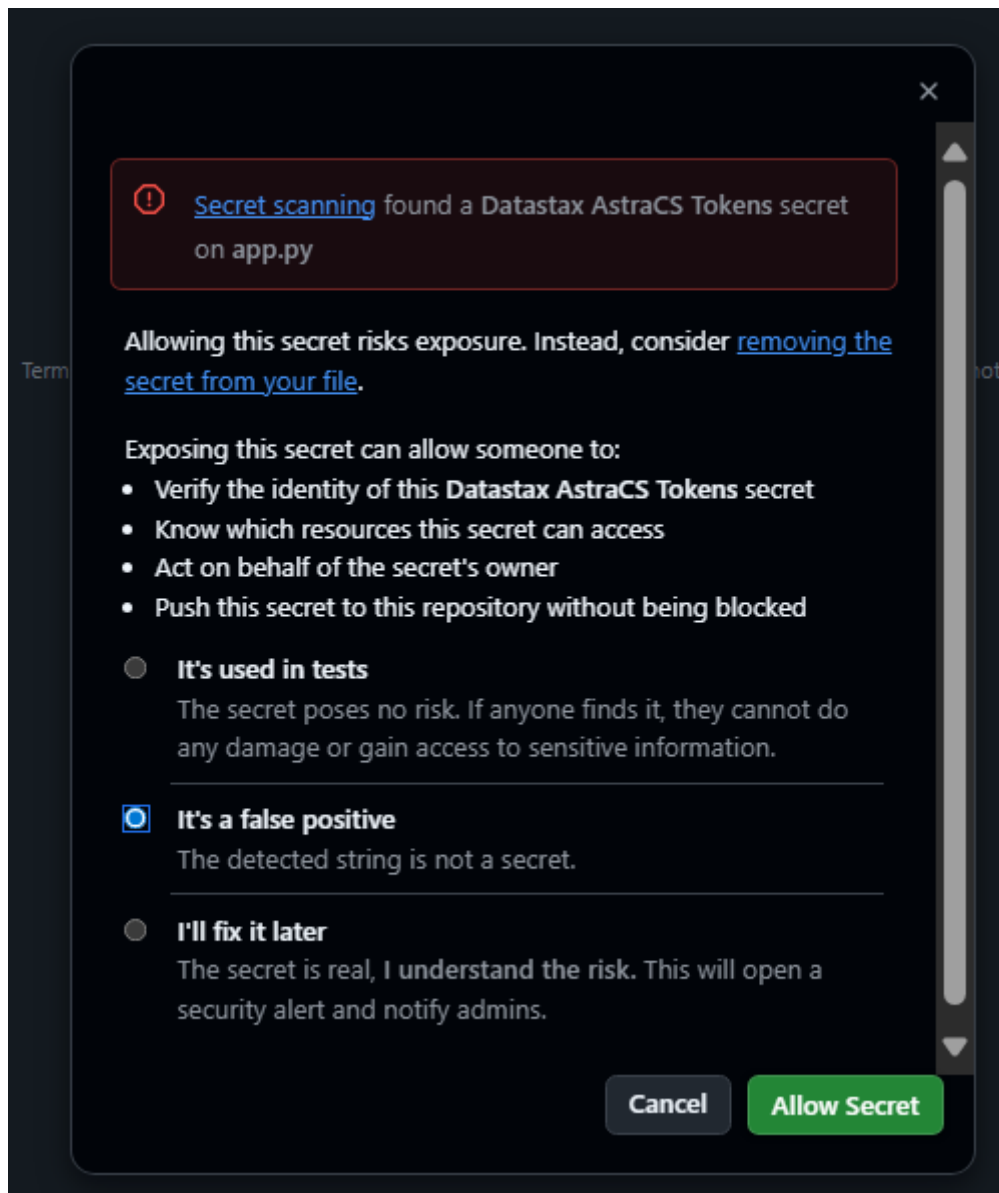


Now upload your code on github

Open github







←↻🔒https://share.streamlit.io

bhupeshther003

My apps

My profile

Explore

Discuss

Create app

bhupeshther003's apps

real_time_sign_project · main · app.py

🌙🌐⋮

weather-api-project · main · main.py

🌙🌐⋮

Get started from a template

View all templates →

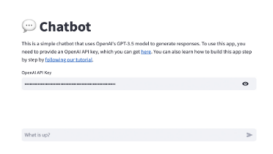
GDP over time



GDP dashboard

View demo

Chatbot



Chatbot

View demo

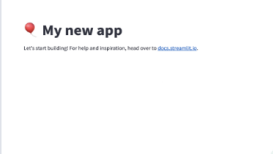
Existing tickets



Support tickets

View demo

My new app



Blank app

View demo

←↻🔒https://share.streamlit.io/new

bhupeshther003

My apps

My profile

Explore

Discuss

← Back

What would you like to do?



Deploy a public app from GitHub

My code is ready on a GitHub repo, and it is totally awesome.

Deploy now



Deploy a public app from a template

I want to see what kind of amazing concoctions you have for me.

Check out templates



Deploy a private app in Snowflake

I want unlimited enterprise-grade apps, with the security of Snowflake.

Start trial →

←↻🔒https://share.streamlit.io/deploy

bhupeshther003

My apps

My profile

Explore

Discuss

Deploy an app

Repository

Paste GitHub URL

bhupeshther003/repo

bhupeshther003/3.11.8-sign-language

bhupeshther003/Astra-DB-Rag-langflow-

bhupeshther003/Blood-Donation-Analytics

bhupeshther003/book

←

↺

https://share.streamlit.io/deploy

🏠 ☆ ⚙️ 🌐 ...

bhupeshther003

▼

My apps

My profile

Explore

Discuss ↗

Repository ⓘ

Paste GitHub URL

bhupeshther003/Astra-DB-Rag-langflow-

Branch

main

Main file path

app.py

App URL (optional)

astra-db-rag-agentic-ai-using-langflow

.streamlit.app

Domain is available

Advanced settings

Deploy

←

↺

https://astra-db-rag-agentic-ai-using-langflow.streamlit.app

Langflow Agentic Ai

Enter your request:

give one month pune trip plan based on current festival

Send to Langflow

✓ Trip Plan Generated

Based on the current festival season, a one-month trip to Pune, India, could be structured as follows:

Week 1: